Bauhaus-Universität Weimar
Faculty of Media
Degree Programme Digital Engineering

Bauhaus-Universität Weimar

Model checking for Self-Replicating Robotic Systems

Master's Thesis

Dmitry Babaytsev (124065)

1. Referee: Prof. Dr. Jan Oliver Ringert
2. Referee: Prof. Dr. Unknown Yet

Submission date: June 31, 2025

Declaration

Unless otherwise indicated in the text or references, this thesis is entirely the product of my own scholarly work.

Weimar, June 31, 2025

Dmitry Babaytsev (124065)

Abstract

Self-replicating robotic systems (SRRS) promise a great potential for space exploration, in-situ construction, and resilient infrastructure, yet critical importance and complex interactions make traditional testing insufficient to guarantee safety or correctness of such system.

This initial research investigates how formal verification and based model checking—can be integrated into the design loop of SRRS to provide provable guarantees of liveness, safety, and functional correctness.

We first survey existing verification techniques and classify state-of-the-art model-checking algorithms with respect to their scalability and suitability for robotics. Building on this analysis, we propose a tool- supported workflow that automatically extracts finite-state abstractions from ROS–Gazebo simulations: mechanical structure is captured from URDF/SDF files, while behaviour is mined from Python ROS nodes and logged execution traces . The resulting models are compiled to NuSMV and checked against temporal -logic specifications that encode key replication properties

An experimental test-bed based on modular robots serves as the reference platform; we evaluate model-checking variants on representative replication scenarios, and try to use counter-example to improve a system. By linking robots simulation to rigorous formal analysis, the work provides a reproducible pathway for checking of next-generation self-replicating robots.

Contents

1 Introduction

A concept of self-replication, has steadily gained interest within the field of robotics as a potential paradigm for enabling autonomous construction, scalability, adaptability, and resilience. Self-replicating robotic systems (SRRS) possess the ability to autonomously build replicas of themselves using available components and resources. These systems hold significant promise for applications in remote environments such as planetary exploration, in-situ resource utilization, and disaster recovery, where manual intervention is limited or infeasible.

However, the complexity and autonomy involved in such systems pose considerable challenges to ensuring their correctness, safety, and reliability.

As SRRS evolve from theoretical constructs to practical experiments, guaranteeing their functional integrity becomes crucial—particularly given their recursive nature and potential to affect their environments in unanticipated ways. Traditional testing approaches are not enough when dealing with critical applications,vast state spaces and non -deterministic behavior inherent in self-replicating systems. In response to these challenges, formal verification methods, and in particular model checking , have emerged as powerful tools for rigorously analyzing system behaviors against formal specifications.

Formal methods offer a mathematically grounded approach to verifying that a system adheres to desired properties such as liveness, safety, and correctness.

This research explores the integration of formal verification into the design and analysis of self-replicating robotic systems. The goal is to bridge the gap between the physical nature of replication in robotics and the abstract frameworks used in model checking.

The structure of this work is as follows. Section 2 provides background on formal verification techniques, model checking strategies, and existing self-replicating robotic frameworks. Section 3 outlines the core research questions for the thesis. Section 4 describes the methodology and tools planed to use for robotic simulations, and presents an approach for experimental validation.

Together, these sections aim to contribute a formal framework for analyzing self-replicating robotic systems, supporting their future deployment in safety-critical and autonomous environments.

2 Background

2.1 Formal verification overview

Formal Methods refers to mathematically rigorous techniques and tools for the specification, design and verification of software and hardware systems.
Expanding this definition Formal methods can be used for formal description of the system (specification) on some level of abstraction, program synthesis (design) according to specification and verification of a system to prove that model of system under consideration satisfies some properties.

Formal verification techniques can be thought of as comprising three parts [Mic04]:

- a framework for modelling systems, typically a description language;
- a specification language for describing the properties to be verified;
- a verification method to establish whether the description of a system satisfies the specification.

These methods and techniques of verification include:

1. SAT solving - operate on propositional logic
2. SMT solving (and tools like Z3) extend this to first-order logic, integrating theories such as arithmetic or arrays.
3. Relational model finding (e.g., Alloy)
4. Model checking (e.g., NuSMV) exhaustively explore state-space against CTL/LTL specifications.
5. Interactive theorem provers (e.g., Coq/Rocq, Isabelle/HOL)

Approaches to verification can be classified according to the following criteria [Mic04] pages 172-173:
Proof-based vs. model-based
In a proof-based approach, the system description is a set of formulas Γ (in a suitable logic) and the specification is another formula ϕ. The verification method consists of

trying to find a proof that $\Gamma \vdash \phi$. In a model-based approach, the system is represented by a model M for an appropriate logic. The specification is again represented by a formula ϕ and the verification method consists of computing whether a model M satisfies ϕ ($M \models \phi$). This computation is usually automatic for finite models. Logical proof systems are often sound and complete, meaning that $\Gamma \vdash \phi$ (provability) holds if, and only if, $\Gamma \models \phi$(semantic entailment) holds, where the latter is defined as follows: for all models M, if for all $\psi \in \Gamma$ we have $M \models \psi$, then $M \models \phi$. Thus, we see that the model-based approach is potentially simpler than the proof-based approach, for it is based on a single model M rather than a possibly infinite class of them.

Degree of automation.
Approaches differ on how automatic the method is; the extremes are fully automatic and fully manual. Many of the computer-assisted techniques are somewhere in the middle.

Full- vs. property-verification.
The specification may describe a single property of the system, or it may describe its full behavior. The latter is typically expensive to verify. Intended domain of application, which may be hardware or software; sequential or concurrent; reactive or terminating; etc. A reactive system is one which reacts to its environment and is not meant to terminate (e.g., operating systems, embedded systems and computer hardware).

Pre- vs. post-development.
Verification is of greater advantage if introduced early in the course of system development, because errors caught earlier in the production cycle are less costly to rectify. (It is alleged that Intel lost millions of dollars by releasing their Pentium chip with the FDIV error.

2.1.1 SAT solvers

Propositional logic - consider propositions and relations between them and constructions of argument based on them.

Propositions - facts or statements about the world that can be true or false . Also defined as atomic propositions or Atoms. Some propositions can be described in natural language as atomic or indecomposable declarative sentences) [Mic04].

Atomic propositions can be connected to each other with logical connectives and compose propositional formulas. The most common connective relations are:

- Negation $\neg$
- Conjunction $\wedge$

- Disunction $\vee$
- Implication $\rightarrow$

The Backus Naur form (BNF) for propositional logic well-formed formula:

$$\phi ::= p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi)$$

where p - is any atomic proposition.

Based on these we can construct a system of reasoning: from some set of formulas we can infer (prove) another formula that can be true or false:

$$\phi_1, \phi_2, \phi 3, ...\phi_n \vdash \psi$$

Such formulas called sequents, where on the left side there is a set of formulas called premises and on the right side - a formula called conclusion

Another relation between logical formulas is semantic entailment:

$$\phi_1, \phi_2, \phi 3, ...\phi_n \models \psi$$

it means that if propositional logic formulas $\phi_1..\phi_n$ evaluate to True, then formula ψ also evaluates to True.

The formula ϕ is valid if for all valuations of atomic propositions in formula it valuates to True:

$$\models \phi$$

SAT solver is a program which aims to solve the Boolean satisfiability problem (SAT), where the Boolean satisfiability problem sometimes called propositional satisfiability asks whether there exists an interpretation that satisfies a given Boolean formula.

Given formula ϕ is **satisfiable** if it has a valuation in which it valuates to True.:

In other words, it asks whether the formula's variables can be consistently replaced by the values TRUE or FALSE to make the formula evaluate to TRUE. If this is the case, the formula is called satisfiable, else unsatisfiable.[Mic04].

SAT problem is NP-complete, as a result, only algorithms with exponential worst-case complexity are known. In spite of this, efficient and scalable algorithms for SAT were developed during the 2000s, which have contributed to dramatic advances in the ability to automatically solve problem instances involving tens of thousands of variables and millions of constraints.

SAT solvers usually begin by converting a problem to conjunctive normal form (CNF). **Conjunctive normal form (CNF)** - as a conjunction of clauses, where each clause is a

disjunction of literals. Literal is an atomic proposition or a negated atomic proposition. Conjunctive normal form allows easy check of validity of a formula which otherwise take times exponential to number of atoms.

2.1.2 SMT

Propositional logic is limited to express relations between logical components other than equivalent to natural language: *not, and, or, if..then* and declarative sentences that can only be True or False.

In order to be able to operate with finer logical structures we need to enrich the language. The necessity to solve this problem lead to design of Predicate logic or first-order logic.
In First-order logic we introduce:

- Variables that represent individual objects.
- Functions $\mathcal{F}$ that refer to objects.
- Predicates $\mathcal{P}$ - symbols that semantically represents some property or relation.
- Quantifiers: universal $\forall$ and existential $\exists$.

The Backus Naur form (BNF) for predicate logic formulas:

$$\phi ::= P(t_1, t_2, ...t_n) \mid (\neg\phi) \mid (\phi \vee \phi) \mid (\phi \wedge \phi) \mid (\phi \rightarrow \phi) \mid (\forall x \phi) \mid (\exists x \phi)$$

where,
t - called Terms and can be variable, constants or functions on terms

$$t ::= x \mid c \mid f(t, ...t)$$

Atomic formulas are predicates applied to terms: $P(t_1, t_2, ...t_n)$

Variables can be free (supplied from outside the formula) or bounded by some quantifier.

Unlike propositional logic, in predicate logical we cannot just evaluate formulas based on given valuation by assigning True or False to the variables. We require a model of all function and predicate symbols involved.
A model M for a first-order logic contains:

- Domain A is a non-empty set of individuals, mapping each variable to a value in A
- Set of functions where n-ary function is interpreted as a concrete function $f^m : A^n \to A$.
- Set of predicates n-ary predicate is interpreted as a relation $P^m \subseteq A^n$

Truth in a model is defined recursively:

$$M \vDash P(t_1, ..., t_n) iff f(t_1, ..., t_n) \in P^m$$

$$M \vDash \text{forall } x\phi \text{ iff for every a } \in A, M \vDash [x \to a]\phi$$
$$M \vDash \text{exists } x\phi \text{ iff for some a } \in A, M \vDash [x \to a]\phi$$

A sentence has no free variables, so its truth value depends solely on M.

We write $\Gamma \vDash \psi$ (semantic entailment) when every model that makes all formulas in Γ true also makes ψ true.

Soundness and completeness ensure $\Gamma \vdash \psi$ (provability) exactly when $\Gamma \vDash \psi$, but the two perspectives complement each other: proofs are constructive, while counter-models establish non-entailment.

2.2 Model checking

Model checking is based on temporal logic. The model in temporal logic described as a system with few states that can change with time. The formula is not statically true or false in a model, as it is in propositional and predicate logic, it can be true in some states and false in others, the static notion is replaced by a dynamic one [Mic04].

In model checking, the models M are transition systems and the properties ϕ are formulas in temporal logic. To verify that a system satisfies a property, we must do three things:

1. model the system using the description language of a model checker, arriving at a model M;
2. code the property using the specification language of the model checker, resulting in a temporal logic formula ϕ;
3. Run the model checker with inputs M and ϕ.

Model checking is a core approach in Formal verification that involves an algorithmic search of the state space of a system model to check if a given property (specification) holds, where:
Model- is an abstract representation of the system (often a state-transition graph or a program) capturing its possible states and transitions. The model can be described in a modeling language (like Promela for SPIN, or as a transition system in a tool like NuSMV).
Specification- is a formal property that the model should satisfy, commonly expressed in a temporal logic such as LTL or CTL. For example, an LTL specification might state that "whenever request happens, eventually response occurs" or a CTL property might require that "in all execution paths, a certain error state is never reached."

2.2.1 Classification of Model checking methods

Model checking can be classified based on time concept into: **linear-time temporal logic** and **branching time logic.** [Mic04]

Linear-time temporal logic

Linear-time temporal logic (LTL) considers time as a set of paths, where each path is a sequence of time instances [Mic04]. LTL has the following syntax in Backus Naur form:

$$\phi ::= T \mid \bot \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi)$$
$$\mid (X\phi) \mid (F\phi) \mid (G\phi) \mid (\phi\ U\ \phi) \mid (\phi\ W\ \phi) \mid (\phi\ R\ \phi)$$

The connectives X, F, G, U, W, R called temporal connectives, because they describe relations between states in different moments of time. The meaning of these connectives described in Table 2.2.1

In LTL, model can be represented as a transition system [SL16].
Transition system T:

$$T = (S, \xrightarrow{a}, L, s)$$

X	Next	ϕ must hold in the very first successor state on the chosen path.
F	Finally ($F\phi$)	ϕ must hold at some future state on the path (i.e., "eventually ϕ").
G	Globally ($G\phi$)	ϕ must hold at every state on the entire path ("always ϕ").
U	Until ($\phi U \psi$)	ψ must be true at a future state, ϕ must stay true at every state before that one.
W	Weak until ($\phi W \psi$)	either ($\phi U \psi$) holds or ϕ stays true forever if ψ never becomes true.
R	Release ($\phi R \psi$)	ψ must hold up to and including the first state where ϕ becomes true; if ϕ never becomes true, ψ must hold forever (the dual of Until).

Table 2.1: LTL temporal connectives

where:
S - set of states, called the space states of T,
$\xrightarrow{a}$ - transitions (binary relation of states), associated with some action $a \in A$
L - labelling function that reflects states to propositional atoms
s - initial state of T, can be omitted if we want to describe only global structure.
Sometimes if we consider only one action label we can omit labeling transition: $\rightarrow$.
A path π in a transition system T is a finite or infinite sequence of states and (optionally) actions which transform each state into its successor: $s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} ...$

Trace or computation in a transition system is a finite or infinite sequence of state descriptions along the path: $L(s_0) \xrightarrow{a_0} L(s_1) \xrightarrow{a_1} ...$

Branching temporal logic (CTL)

Branching temporal logic represents time as a tree rooted in present and branching into the future, the other name is Computation Tree logic (CTL). LTL implicitly quantifies universally over paths [SL16]. Therefore, properties which assert the existence of a path cannot be expressed in LTL. We can solve this problem by considering the negation of the property in question. For example to check whether there exists a path from s satisfying the LTL formula ϕ, we check whether all paths satisfy $\neg\phi$; a positive answer to this is a negative answer to our original question, and vice versa. We used this approach when analysing the ferryman puzzle in the previous section. However, properties which mix universal and existential path quantifies cannot in general be model checked using LTL approach, because the complement formula still has a mix. Branching-time logics solve this problem by allowing us to quantify explicitly over paths.

CTL has the following syntax in Backus Naur form:

$$\phi ::= T \mid \bot \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi)$$

$$\mid (AX\phi) \mid (EX\phi) \mid (AF\phi) \mid (EF\phi) \mid (AG\phi) \mid (EG\phi) \mid A[\phi\ U\ \phi] \mid E[\phi\ U\ \phi]$$

New introduced quantifiers A and E mean:
$A\phi$ - formula ϕ for all paths
$E\phi$ - exists a path that formula ϕ
Connectives X,F,G cannot occure without being preceded by A or E.
Weak until W and release R are usually not included in CTL, but can be derived.

Temporal logic (CTL*)

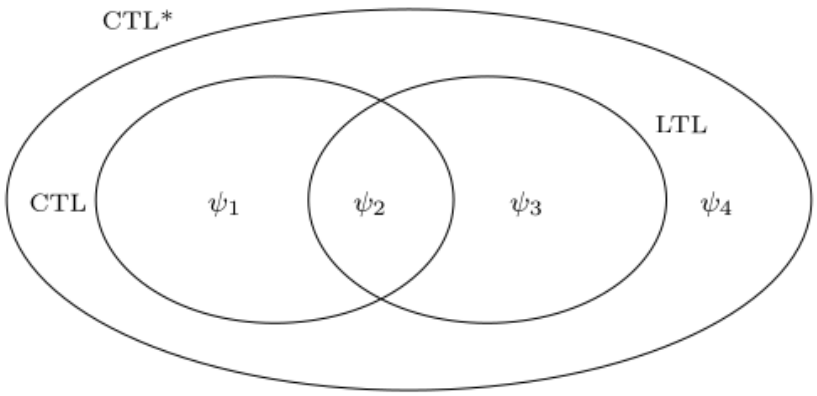

Figure 2.1: LTL, CTL and CTL* logic relation [Mic04]

Statistical Model Checking (SMC)

Some systems have huge or continuous state spaces (e.g. complicated analog components, or software with random timing), or they exhibit probabilistic behavior (randomized protocols, hardware failures) where one is interested in quantitative properties (like "the request will be served within 1s with probability at least 0.99"). Statistical Model Checking (SMC) is an approximate verification technique that uses simulation and statistical methods instead of exhaustive exploration [YS02].

SMC treats the model checking problem as a statistical inference problem [You06]. It executes a number of simulations (random or guided) of the system. Checks whether

each simulation trace satisfies the property ϕ (which often is a bounded temporal logic property like “within T time units, something happens”). Based on the outcomes, it uses statistics (like hypothesis testing or estimation with confidence intervals) to infer with high confidence whether the property holds with required probability.

SMC was pioneered by Younes in the 2000s [YS02]. There are two main modes: hypothesis testing (e.g. "check if the probability $P(property) > p$ with confidence 95%") and estimation (estimate the probability itself within some error bound).

They adopt the continuous stochastic logic (CSL) as our formalism for expressing probabilistic real-time properties of discrete event systems. CSL has previously been proposed as a formalism for expressing temporal and probabilistic properties of continuous-time Markov chains (CTMCs) by Aziz et al. [Azi+96].

A CSL formula is either a state formula or a path formula.

The Backus Naur form (BNF) for CSL logic well-formed formula:

State formulas:

$$\phi ::= \text{true} \mid a \mid \neg\phi \mid \phi \wedge \phi \mid P_{\leq\theta}(\rho)(\phi)$$

Path formulas:

$$\rho ::= X\phi \mid \phi U_{\leq t}\phi$$

Where ϕ is a state formula and ρ is a path formula.

Tools like PRISM and its SMC extension, described in [KNP11], STORM [Deh+17], or PLASMA Lab [LST16] implement statistical model checking for various kinds of stochastic systems.

The obvious advantage is that SMC sidesteps the state explosion by not exploring the full state space – instead, it samples paths. This makes it scalable to systems with enormous or even infinite state spaces, as long as we can simulate the system. It is also applicable to systems with continuous dynamics or very large numbers of states where traditional model checking is infeasible . Another benefit is ease of implementation: if there is a simulator for your system, adding a statistical checker on top is easier than building a full model checker. SMC naturally supports probabilistic properties – e.g., verifying properties of a Markov Chain or Markov Decision Process by sampling behaviors and using results like Chernoff or Hoeffding bounds to determine if the property likely holds. Because it uses actual simulations, SMC can handle very complex nonlinear or black-box system aspects (treating them as black boxes that just produce simulation traces).

The trade-off is that SMC provides a confidence result, not a guarantee. It is probabilistic verification: it might say “with 99% confidence the property holds with probability

≤ 0.97". There's a small chance of error in the verification result (false positives/negatives bounded by the confidence level). Also, SMC may require a large number of simulations to reach high confidence, especially for rare events (if the property failing is a rare event, one might need many samples to observe it even once). Rare property verification is a known challenge – advanced importance sampling or rare event simulation techniques might be needed in such cases.

SMC can't truly prove a property in the mathematical sense – it can only give statistical evidence. If a property almost always holds but has a corner-case failure, SMC might miss it if that scenario has very low probability and isn't sampled. Thus, it sacrifices completeness. Another limitation is that SMC typically handles quantitative properties (often expressed in probabilistic temporal logics like PCTL or BLTL) – it's not usually used for pure logical properties that are either true or false; it's more for properties like "the probability of an event is at least p".

Runtime Verification and Monitoring

Runtime verification (RV) is a technique that involves monitoring a running system (or execution traces produced by it) against a formal specification [HSZ14]. Unlike the other methods discussed, which are largely offline (analyzing a model mathematically), runtime verification is applied during execution. It can be seen as a lightweight formal method, sitting somewhere between testing and model checking: you instrument the system with monitors, and these monitors check whether the execution is satisfying the desired properties. If a violation is detected, it can be logged, or in some cases, a recovery action can be taken.

Key components of runtime verification include:
- A set of monitors or runtime assertions derived from the formal specification (often temporal logic formulae or automata on traces).
- An instrumentation of the system to report events to the monitors.
- The monitors run in parallel with the system (or on logged traces)
and detect property satisfaction or violation on the fly.

For example, if you have a property "whenever interrupt is raised, service routine runs within 100ms," a runtime monitor can watch for interrupts and check timing until service routine is called, flagging an error if the 100ms deadline is missed.

Runtime verification avoids the state explosion problem entirely, because it doesn't explore all behaviors — it only checks the actual behaviors that occur. Runtime verification avoids the complexity of traditional formal verification techniques (like model checking and theorem proving) by analyzing only one or a few execution traces and by working directly with the actual system. This means it scales well to very complex

systems, since you're just running the system (or a high-fidelity simulation of it) and the overhead is only in checking the property on that run. It also has the advantage of checking the real system (no abstraction or modeling error, since the code that runs is what's verified against the property , at least for that run). It can catch errors in scenarios that were not anticipated by designers as soon as they occur in the field or during testing, thus serving as a powerful debugging and validation aid. Runtime verification can be used continuously in deployed systems for critical properties (e.g., a monitor that ensures no unauthorized file access operations occur, and triggers an alarm if they do, providing a layer of security monitoring). Another plus is that runtime verification can sometimes detect issues that static model checking might miss due to environment assumptions. At runtime, the real environment provides inputs and the monitor can see violations under real conditions, including those hard-to-model aspects.

The fundamental limitation is that runtime verification is not exhaustive. It can only inspect the executions that actually happen (or a sample of them). If a particular bug does not manifest in the observed runs, RV cannot detect it. It provides no guarantee that "no bug exists" — only that "no bug was observed in the runs we monitored". In other words, it lacks completeness/coverage , unlike model checking which (for a model) covers all behaviors by design. Another challenge is performance overhead: monitors consume resources, and if properties are complex (say, specified in LTL, which might require monitoring a suffix of the trace), the runtime checking might slow the system or require buffering events. However, in many cases monitors can be optimized or run on a separate core, and the overhead is manageable (reports of a few percent overhead for simple monitors are common). Additionally, writing good monitors (formal properties) and instrumenting the system requires effort. It's easier than full verification, but still requires formal specification skills. If one instrument too much or monitors too many properties, the overhead can become significant, or the system 's real-time behavior might be perturbed.

Runtime verification is often used in safety-critical systems as an additional runtime safety net. For instance, in aerospace or automotive software, one might include monitors for conditions like "no two actuators turn on simultaneously" or "if sensor reading exceeds threshold , system enters safe mode within 1 second," etc. NASA has explored RV for spacecraft software monitoring during missions [Sla+24](since you cannot model check the entire flight code with all analog intricacies, but you can monitor key properties as the mission progresses). Runtime Verification has also become popular in security (e.g., monitoring program execution for compliance with security policies). For example, a monitor can watch system calls to detect a sequence that matches a known attack pattern or a violation of a security automaton. Another domain is testing: during testing, using runtime monitors can greatly increase the chance of noticing an issue. Instead of manually inspecting outputs , formal monitors can rig-

orously check that each test execution satisfies the expected temporal properties. This is often called “monitoring oracles” in testing – formal specifications serve as test oracles at runtime. In summary, runtime verification does not replace model checking but complements it. Where model checking cannot be applied (due to scale or undecidability), or in deployed scenarios where you want continual assurance, RV provides a way to leverage formal specifications dynamically. It trades completeness for scalability and immediacy.

2.3 Self-replicating robotic systems

2.3.1 Self-replication introduction

Self-replication is the process by which a dynamical system constructs an identical—or very similar—copy of itself. According to Sipper, replication is an ontogenetic, purely developmental phenomenon: it involves no genetic- or code-modifying operators and yields an exact duplicate of the parent organism.

Replication must therefore be distinguished from reproduction, a phylogenetic, evolutionary process that employs genetic operators (e.g., crossover and mutation in biology) and ultimately generates diversity and evolution [Sip98].

This study focuses exclusively on replication and self-replication, code-altering and evolutionary processes are beyond its scope.

Self-assembly refers to the autonomous organization of components into patterns or structures without human intervention. In robotics, this term may describe either (i) a kinematic machine that manipulates discrete parts to build an assembled copy of itself (kinematic self-replication) or (ii) a collective of mobile robots that can autonomously connect, disconnect, and reconfigure themselves (swarm robotics and robotic self-reconfiguration [Tan+20]).

When the replication process is controlled solely by the system being reproduced, it is called a self-replicating system [LC07]. In contrast, if the replication process requires external elements that actively control and manipulate resources and direct the process, it is simply a replicating system. A biological analogue is a virus, which replicates by exploiting a host cell.

A self-replicating machine or self-replicating robotic system (SRRS) is thus an autonomous robot capable of manufacturing a copy of itself from raw materials, or from minimal possible units related to considered environment, thereby exhibiting self-replication in a manner analogous to that observed in nature.

As shown by Chirikjian in [Chi22] "the basic difficulty is that in order to close the self-replicating loop, there must be a balance between the overall system complexity and the ability of the system to handle the simplest inputs possible. Generally speaking, the more complex the system is, the more capable it is to assemble the simplest parts, or even harvest raw materials. But when the system is very complex, it then requires more to reproduce. For example, if a robot needs microprocessors and a 3D printing system needs a high-energy laser, then producing those from in situ resources becomes an enormous challenge." And "One way around that problem is to focus on production

of mechanical parts in situ such as structural members for robots, factories, and habitats, and to send the relatively light-weight “vitamin” components such as computers, lasers, and some chemical reagents to be considered as inputs to the system rather than as items to be replicated. In this way, a practical version of in situ resource utilization (ISRU) can be achieved by reclassifying inputs. Moreover, developing systems that are depen- dent on some inputs that the systems cannot manufacture from scratch is both more akin to living systems that require specific nutrients, and is also safer than developing self-replicating systems that could continue to replicate without the imposition of resource bottlnecks. Without such intentional bottlenecks there could be fears of things running amuck for generations."

2.3.2 Classification of Self-replication systems

Type-related replication

Building upon the foundational work of Jones [Jon21] and works of Lee and Chirikjian , self-replicating systems can be systematically classified according to their type-related replication architecture into three primary categories:

Centralized replication system where robots can produce only robots without replication functionality as is shown on Figure 2.2a.
The centralized approach has several distinct advantages: it provides precise control over population growth, ensures quality consistency through standardized manufacturing processes, and reduces resource competition among robots. However it also introduces significant vulnerability through single points of failure - if the replicating robots are damaged or destroyed, the entire system’s reproductive capacity is compromised.

Additionally, the concentration of replication resources that have to be located or transported near the replicating robots may create bottlenecks that limit the system’s expansion rate and adaptive responsiveness.

Decentralized replication represents a system architecture where every robot within the system possesses full replication capabilities, see Figure 2.2b. This distributed approach is inspired by cellular division processes, where each entity contains the complete blueprint and machinery necessary for creating identical copies of itself.

The decentralized model excels in resilience, scalability and speed of self-replication. The system can continue expanding even if individual robots are eliminated, as each surviving unit has complete reproductive autonomy. This architecture facilitates rapid colonization of new environments and provides redundancy against catastrophic failures. However, decentralized systems face challenges in coordination and control for

some systems, possible uncontrolled exponential growth, and consistent quality across independently operating replication processes.

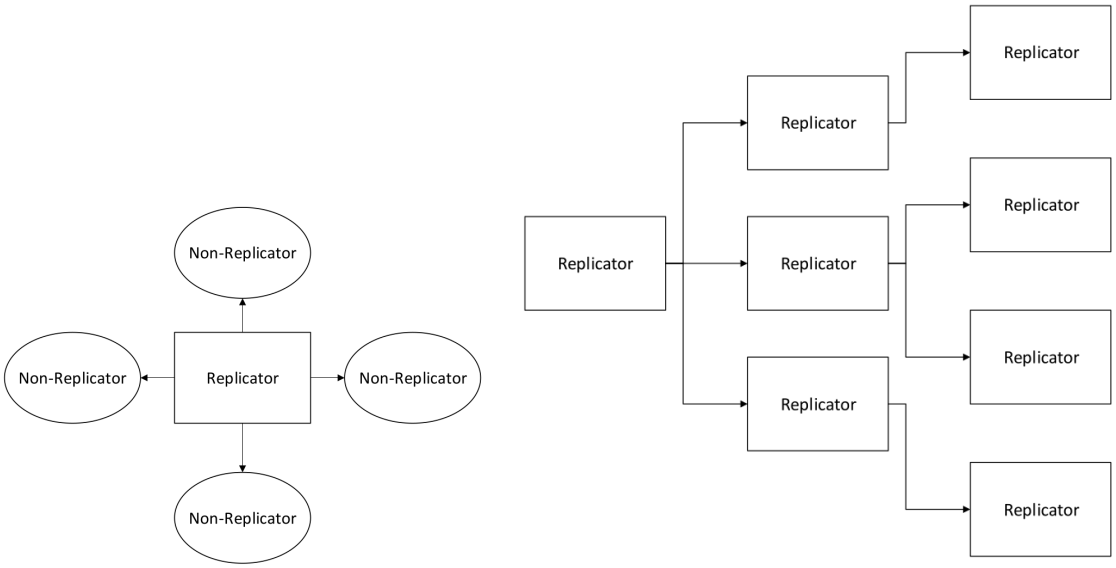

Figure 2.2: Centralized and decentralized replication [JS21]

Hierarchical replication architectures represent a hybrid approach that combines elements of both centralized and decentralized strategies. In these systems, robots with replication capabilities can produce two distinct categories of offspring: specialized robots lacking replication ability (similar to centralized systems) and fully autonomous robots capable of self-replication (similar to decentralized systems), as is shown on Figure 2.6.

This multi-tiered approach enables dynamic adaptation to environmental conditions and mission requirements. The system can produce specialized worker robots for specific tasks while simultaneously maintaining a population of replicator robots to ensure continued expansion and redundancy.

The hierarchical model allows for optimized resource allocation, where replication-capable robots can focus on expansion while non-replicating robots specialize in operational tasks, environmental sensing, or resource collection.

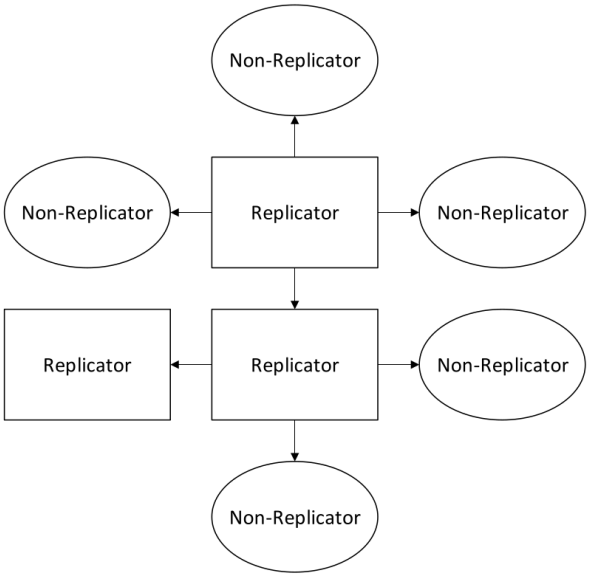

Figure 2.3: Hierarchical replication [JS21]

Population Composition Classification

The composition of robot types within an SRRS fundamentally influences system behavior, specialization potential, and adaptive capacity. This classification dimension addresses the diversity of robotic types operating within the system.

Homogeneous Systems Homogeneous SRRS consist of robotically identical units, where all entities share the same physical design, computational capabilities, and behavioral programming. This uniformity creates predictable system dynamics and simplifies coordination protocols, as all robots operate with identical sensor suites, actuator capabilities, and decision-making algorithms.

The homogeneous approach offers advantages in replication process consistency, simplified communication protocols, and predictable emergent behaviors. Resource allocation becomes straightforward when all units have identical requirements, and system-wide updates or modifications can be uniformly applied. However, homogeneous systems may struggle with task specialization and environmental adaptation, as the lack of diversity limits the system's ability to optimize for varied operational requirements.

Heterogeneous Systems Heterogeneous SRRS incorporate multiple distinct robot types, each potentially specialized for specific functions, environmental conditions, or operational roles. This diversity enables flexible division of labor, where different

robot types can be optimized for exploration, resource gathering, construction, maintenance, or replication tasks.

The heterogeneous model provides enhanced adaptability and operational efficiency through specialization. Different robot types can be designed with complementary capabilities, creating synergistic effects that exceed the performance of individual units. This approach enables the system to respond more effectively to complex environmental challenges and mission requirements.

However, heterogeneous systems require sophisticated coordination mechanisms to manage inter-type communication, resource distribution, and collaborative decision-making across diverse robotic platforms.

Interaction with environment or external system

Another classification of self-replicating robotic systems can be made based on the degree of autonomy and reliance on external systems, as it described in [LC07]. A fundamental distinction is made between passive and active self-replication:

Passive self-replicating systems do not possess inherent mechanisms to actively reproduce themselves. However, given an appropriate environment and the right configuration of components, replication can still occur. A canonical example of this is the Penrose block replicators, which demonstrate self-replication through mechanical constraints and spatial arrangement without any active control [Pen58].

Active self-replicating systems, on the other hand, are equipped with the functional capacity to carry out the replication process. These systems may still rely on passive environmental structures (such as fixtures or gravity) to assist in replication, but are actively engaged in constructing replicas of themselves.

Interaction with environment or external system, alternative classification

A more detailed and operationally useful classification is introduced by Suthakorn, Zhou, and Chirikjian [SZC02]. This framework emphasizes how SRRS interact with their environments and whether replication is accomplished directly or through intermediate stages. Based on this framework, all SRRS are categorized into two primary classes:

Directly replicating SRRS - in which a robot is capable of constructing an exact copy of itself in a single generational step. These systems can be further divided into the following subcategories, based on their reliance on fixtures and replication strategy:

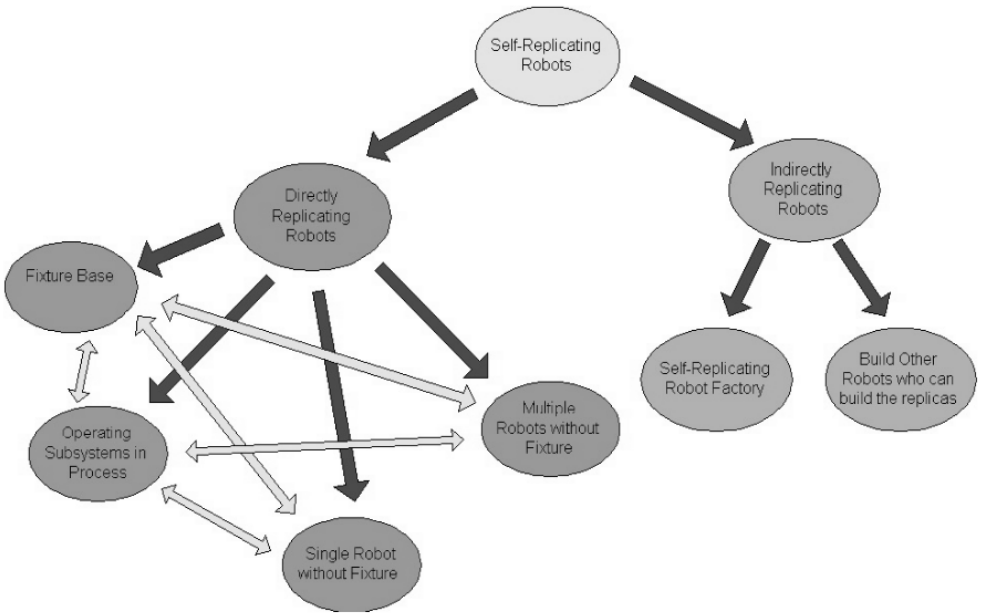

Figure 2.4: Directly and indirectly replicating robots classification [SKC03]

Directly Replicating Robots can be further classified into following groups:

- Fixture-Based Group The self-replicating robots in this group depend on external fixtures in order to complete the self-replication process. Passive fixtures are able to assist in this because of the shape constraints that they impose. In some other cases, to unify subsystems, push-pull fixtures are helpful as well.
- Operating-Subsystem-in-Process Group In this group one or several subsystems of the replica can operate before the replica itself is fully assembled. These subsystems are able to assist the original self-replicating robot during the assembly of the replica.
- Single-Robot-Without-Fixture Group In this group only one robot is used to finish the self-replication process. Thus, the robot in this group depends only on the available environment. Usually, the complexity of the subsystems or the number of subsystems in the replica is very low for this group.
- Multi-Robot-Without-Fixture Group In this group more than one robot works together in the self-replication process without the assistance of fixtures. A major advantage is the reduction of the time required for self-replication. A disadvantage is that there may be interference problems among robots. There are several possible ways that a self-replicating robot can be categorized in two or

three groups mentioned above. The combination of two or three different concepts can be incorporated in a potential design, such as a combining operating-subsystems-in-process with fixture-based robots. More categories are likely to be developed in the subsequent stages of our research in the area of self-replicating robots.

Indirectly replicating - where replication is achieved through intermediate stages rather than a direct one-to-one duplication. The original robot initiates the creation of other robots or constructs a specialized environment that facilitates replication. Indirect replication strategies include:

- Self-Replicating Robot Factory A robot or group of robots constructs a fabrication environment — a factory, that can then autonomously produce replicas of the original. This approach encapsulates the replication process into a fixed infrastructure, enabling high throughput and modular production.
- Intermediate Robot Builders Instead of replicating itself directly, a robot creates new robots that are not replicas, but are instead specialized for manufacturing. These intermediate agents then produce replicas of the original robot. This cascading replication strategy introduces flexibility and specialization but may involve increased system complexity.

2.3.3 Self-replication models

The concept of self-replicating systems originates from the foundational work of Neumann and Burks, who in 1962 proposed the first theoretical model of a self-reproducing automaton [NB62]. This model, developed in the context of cellular automata, formalized the minimal components necessary for a machine to replicate itself.

According to this model, a self-reproducing automaton consists of four key components:
A (constructor): an automatic factory that collects raw materials and processes them into outputs using a specified written instruction,
B (copier): that takes an instruction and copies it,
C (controller): that controls A and B, actuating them alternatively,
D (instructions) A coded description of the automaton's structure and function, acting as its "genetic" information.

$$(A + B + C + D) \rightarrow (A + B + C + D), (A + B + C + D)$$

And the replication process in von Neumann's model, is when the automaton $(A + B + C + D)$ produces another $(A + B + C + D)$.

However, while von Neumann's model is conceptually complete for information-theoretic or logical systems, it exhibits limitations when applied to physical robotic systems. Specifically, it doesnt describe interaction with a physical environment, resource acquisition, resource transformation and management of by-products, energy, and mechanical constraints.

To address these limitations, a more general and physically grounded model was proposed by Lee and Chirikjian [LC07]. In this extended model, the replication process is formalized as a transformation: Lee and Chirikjian proposed broader model, where a replication process can be written as

$$(R, M, E) \rightarrow (R', M', E') \tag{2.1}$$

where:
M is a multiset of available parts to be used for building replicas by an initial system
R is a multiset of an initial functional system to be reproduced. $R \neq 0$; to be a replicating system.
E is a multiset of environmental structures and/or elements involved in the replication process (but not replicated).
R', M', E', the replicated entities, the remaining or altered material and the possibly changed environment respectively.

In this model R describes the whole replicator $A + B + C + D$ from Von Neumann's model.

This transformation can also be expressed functionally as:

$$(R', M', E') = \Phi(R, M, E, t) \tag{2.2}$$

Where Φ denotes the replication function and t is time. This formulation enables modeling dynamic replication scenarios over time and accounts for state changes in materials and environment.

The model explicitly incorporates environmental interaction (E), which plays a critical role in physical self-replication. As proposed by Eno, Mace, Liu, et al. [EML+07], environments can be categorized based on the level of structure and predictability:

- Completely structured environment - in which every environmental structure is fixed at a precise location and there is no modification or change in any environmental structures during the self-replication process. If we define E' is the set of environmental structures after self-replication process: $E = E' \neq \phi$

- Partially structured environment is like a structured environment, but with structures whose locations can be permuted and perturbed by a small random error in pose during the self-replication process. $E \neq E \neq \phi$
- Unstructured environment has no predefined or organized structure and can dynamically change during the self-replication. Can be formally depicted as: $E = \phi$ where ϕ represents a stochastic or unknown environmental process.

2.3.4 Degree of self-replication

As shown in [Chi22]: "In an artificial self-replicating system, if the number of input parts is small, and if each part is itself a complex machine, then the effort involved in the assembly step of self-replication is relatively low compared to the effort of making the input parts. The assembly task in self-replication is then relatively unchallenging in such a scenario. In contrast, if there are many simple (easy to manufacture) parts that need to be assembled by the robot, the relative difficulty is higher. The difference between these two scenarios can be quantified using the concept of the 'degree of self replication'. The highest degree of self replication is when raw materials are harvested by a system and used to produce a replica, since fabrication and assembly are both done by the self-replicating system in that scenario."

In general as shown in [Chi22] degree of self-replication can be abstractly described as:

$$DOSR = \frac{System\ Complexity}{Parts\ Complexity}$$

In 2007 Lee and Chirikjian proposed a measure of degree of replication [LC07]: measure of how much order is created by the assembly process relative to the existing order in the unassembled parts. In this section, we first define a combined measure of system complexity distribution and relative complexity for ARS.

As a simple measure of subsystem complexity, we count the number of active elements in each subsystem. An active element is a moving mechanical part or a fundamental electronic component. Some special parts, such as a chassis and fixed mechanical parts with special purpose (e.g., a passive end- effector), are also viewed as active elements even if they are not moving mechanical parts.

For a robotic system (recall that all robotic systems are active by our definition) consisting of n subsystems, we define the degree of replication for the system, D_s , as

$$D_s = \frac{C_{min}}{C_{max}} \cdot \frac{C_{total}}{C_{ave}} \cdot \frac{1}{C_{ave}} \tag{2.3}$$

where

$$C_{ave} = \frac{1}{n}\sum_{i=1}^{n} C_i$$

$$C_{max} = maxC_1, ..., C_n$$

$$C_{min} = minC_1, ..., C_n$$

$$C_{total} = \sum_{i=1}^{n} C_i + \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} C_{ij}$$

C_i is the number of active elements in M_i
C_{ij} is the number of interconnections between the i-th and j-th subsystems when they are assembled. $C_{ij} = 0$ if the i-th and j-th subsystems are not directly connected by the assembly process.

In formula **??** the first term measures the complexity distribution throughout the subsystems, the second term measures the relative complexity of the total system to the individual subsystems, and the last term penalizes for complex subsystems. To design a robotic system capable of replication from low-level parts, the subsystem complexity should remain low relative to the total system complexity (i.e., high relative complexity).

In addition, we view a system consisting of a few complex parts as more trivial than one composed of many low-complexity parts. A higher value of D_s indicates a more truly replicating system in terms of its complexity distribution and relative complexity. Although the concept of D_s may not generalize to all robotic systems, it is useful as a measure to evaluate robotic replicating systems.

2.3.5 Implementations of kinematic self-replicating robotic systems

Low-complexity self-replication systems from specialized modules

The experimental system developed by Lee and Chirikjian demonstrates active self-replication using six simple electromechanical subsystems [LC07]. The system operates autonomously within a partially structured environment and constructs a functional replica of itself without external control or manipulation. This prototype serves as a proof-of-concept for scalable robotic self-replication using low-complexity parts and embedded environmental cues.

The system is a physical prototype of a self-replicating robot made up of six simple electromechanical subsystems (called M1 to M6) as it shown on the Figure 2.5.

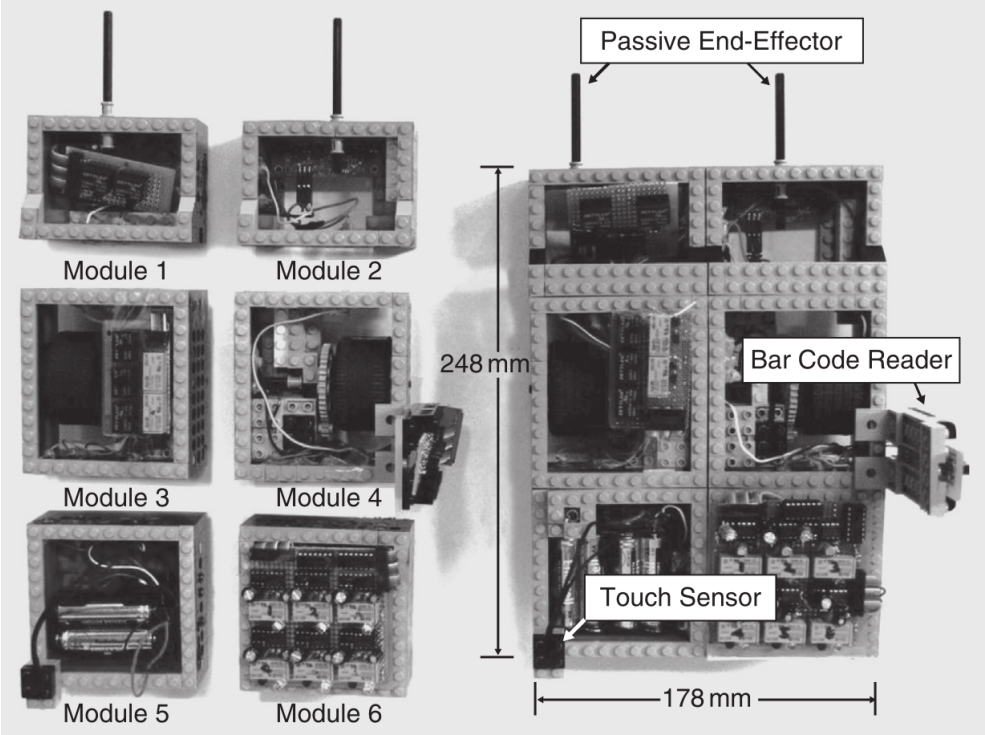

Figure 2.5: Modules (left) and complete robot (right) [LC07]

These modules, when assembled in a specific order, form a complete and functional mobile robot. The purpose of the experiment was to demonstrate that robotic self-replication is possible using relatively low-complexity parts in a partially structured environment—a space that aids replication without directly controlling it. Together, these modules allow the robot to: move along predefined paths, detect locations of components, decide what action to perform and carry and assemble parts.

The robot operates within a partially structured environment—one that contains helpful features, but does not actively participate in the replication. It includes: outer and inner tracks, crossways (branch paths), bar codes, contact codes, sensors, assembly station and wall, used to align the robot during reversing, ensuring precise placement.

The robot has six states, see Figure 2.7 and three distinctive behaviors for each state: forward line-tracking (line-tracking mode), reversing (reversing mode), and turning left while the timer is on (left-turning mode). The replication process begins with the parent robot navigating along a black track using its line-tracking sensors. As it moves, it detects barcodes placed near the modules it needs to collect; each barcode signals the robot to turn and approach a module. Using its passive end-effector, the robot captures the module and transitions to a new state based on contact sensors. It

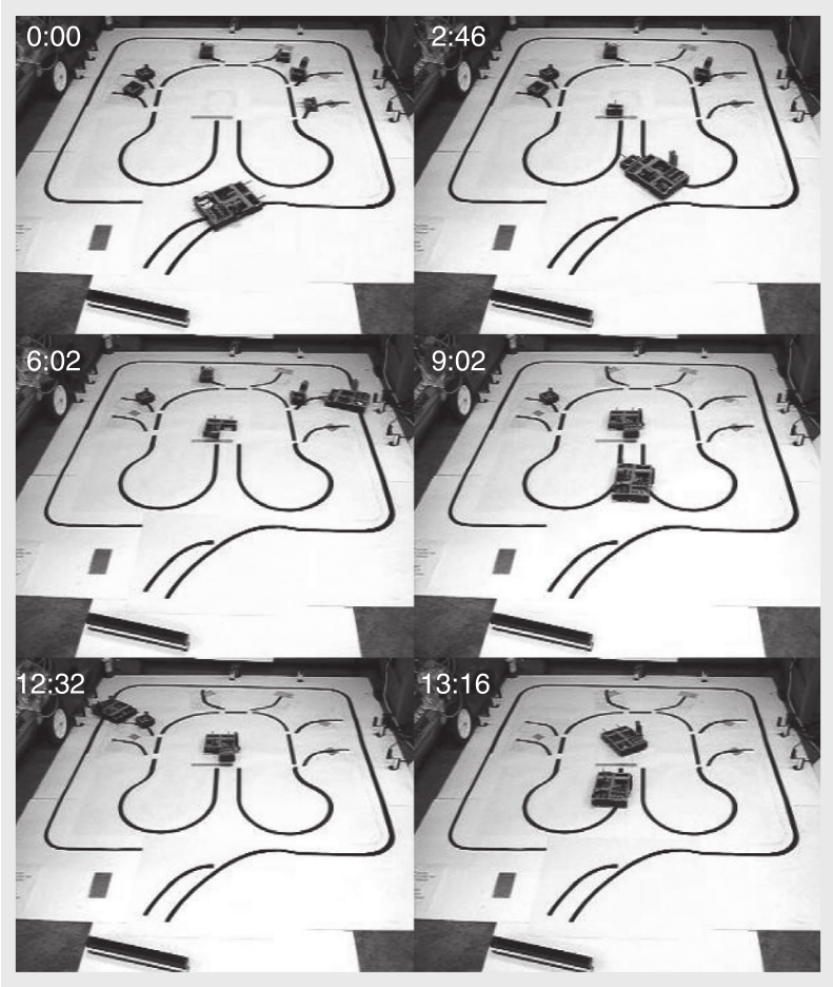

Figure 2.6: The process of self-replication [LC07]

then transports the collected module to a central assembly station marked by a metal line, which triggers the robot to reverse. The robot backs into position and places the module, using a wall as a physical guide for alignment. After delivering the module, it returns to the track and continues this process for the remaining modules. Each time, a new barcode guides it to the next module, and contact codes confirm successful transitions. The modules are assembled in a layered structure, with specific modules forming the base, middle, and top of the robot. Once all six modules are in place and properly connected, the electrical circuit is completed, and the new robot becomes fully functional. The entire process is autonomous and guided solely by the robot's internal logic and environmental cues successfully constructing a working copy of itself without any external control or assistance.

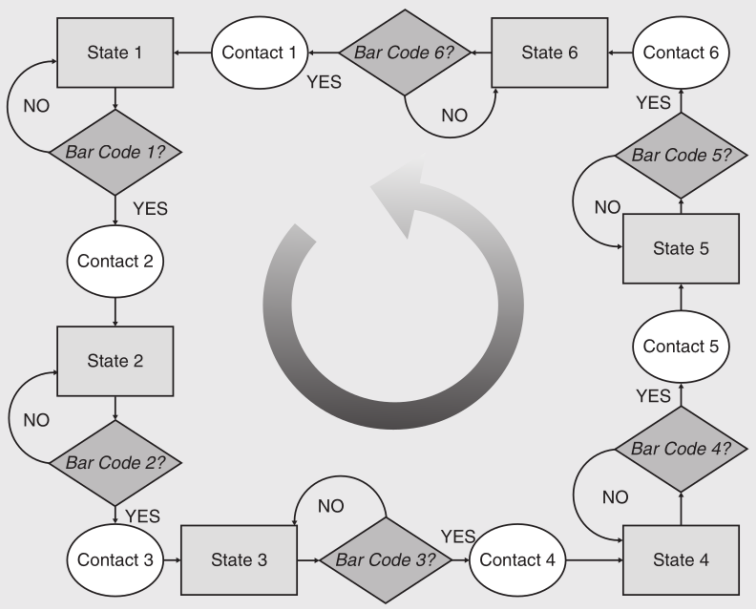

Figure 2.7: State diagram shows behaviors of the robot according to the sensor inputs [LC07].

Using formalas **??** and **??** the system can be represented by

$$R = M_1 + ... + M_6$$

$$M = M_1 + ... + M_6$$

$$E = \{\text{tracks, bar codes, contact codes, wall, metal line}\}$$

, the system becomes

$$R' = \{(M_1 + ... + M_6),\ (M_1 + ... + M_6)\}$$

$$M' = \emptyset$$

$$E' = E.$$

Kinematic self-replication from unified modules

Different authors proposed kinematic or geometrical self-replication based on unified modules. Zykov et al. in 2005 demonstrates physical self-replication using autonomous modular robots [Zyk+05].

These machines are built from identical robotic cubes—each a 10 cm module—that can physically attach, detach, and reconfigure to construct replicas of themselves.

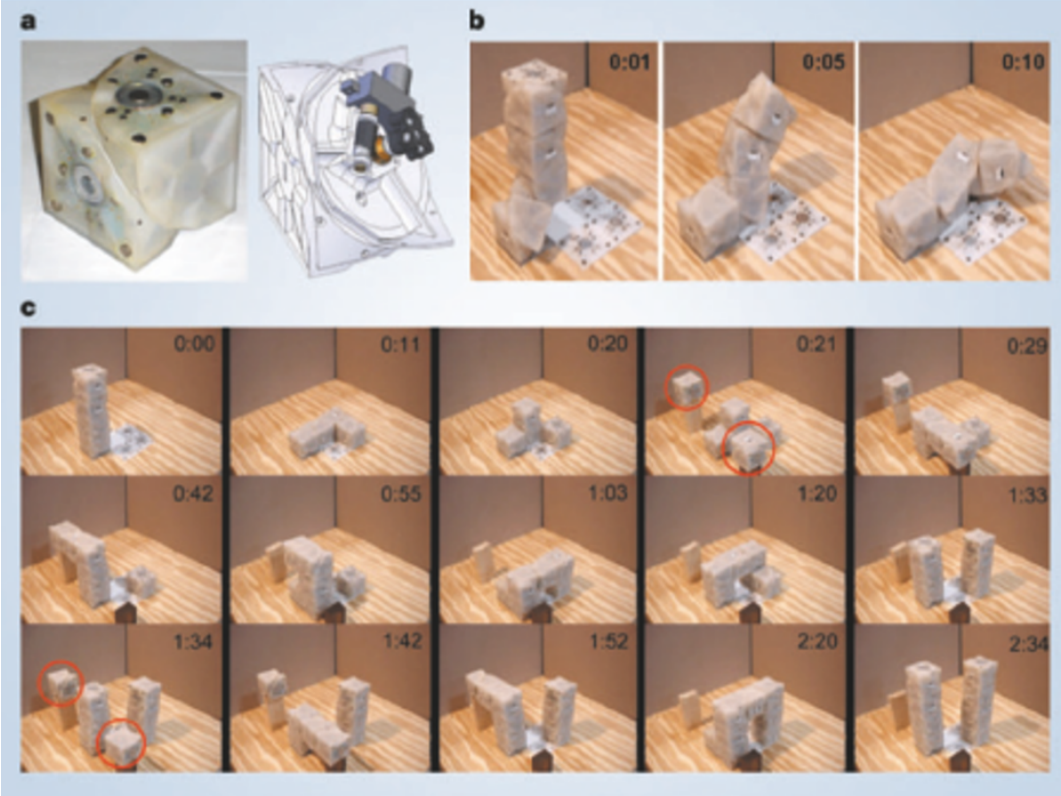

Figure 2.8: Self-replication of a four-module robot [Zyk+05]

Each cube module is split along a diagonal plane and contains internal mechanisms that allow one half to rotate relative to the other, see Figure 2.8a. This ability enables modules to cycle between different configurations by swiveling in few seconds (see Figure 2.8b), allowing connected modules to break apart and reattach in new positions. The modules use electromagnets for selective connection and disconnection, and they receive power and communication signals through conductive face-to-face contacts and a powered baseplate.

The replication process begins with a small robot, composed of four such modules, positioned near two "feeding stations" that contain spare cubes. These cubes are manually replenished, but the entire replication procedure is autonomous. Using their internal control logic and distributed microcontrollers, the robot's modules coordinate motion to pick up new cubes, manipulate them into position, and assemble them into a new, functional robot of the same structure. In one experiment, a four-module robot completed replication in approximately 2.5 minutes (see Figure 2.8), while a simpler three-module robot managed the task in just over one minute. This experiment demonstrates that even simple robotic parts can support mechanical self-replication

so that the resulting system can itself repeat the replication process.

Material-robot systems

The material-robotic system proposed by Jenett et al. in [Jen+19] and further developed by Abdel-Rahman et al. in [Abd+22] consist of passive structural lattice voxels which form a substrate for the locomotion of purpose-built inchworm modular robots build from active voxels, and capable of rearranging and placing additional voxels, see Figure 2.9.

The combination of voxels and robots forms a system, enabling precise assembly of large structures with simple robots through localized registration to the underlying lattice, and also allows of creation of modular robotic toolkit that utilizes active lattice voxels as the primary structural building blocks.

The active voxels which form the basis of the structural-robotic system are a cuboctahedral (Cuboct) unit cell which offers superior structural properties such as high stiffness at low density with geometric characteristics suited for robotic assembly and fabrication. Specifically the flat Cuboct faces enable fully- constrained connections between single-voxel pairs with four points of contact, as well as a useful decomposition for voxel fabrication.

(a) Cell **(b)** Modular robot

Figure 2.9: Material-robot system [Abd+22]

Combining these active voxels with actuators, control, and power enables unique behaviors including robotic self-replication and hierarchical robot assembly. A single

robot, tasked with constructing a large structure, can create a heterogeneous swarm of constructors tailored to maximize material throughput for a given target geometry.

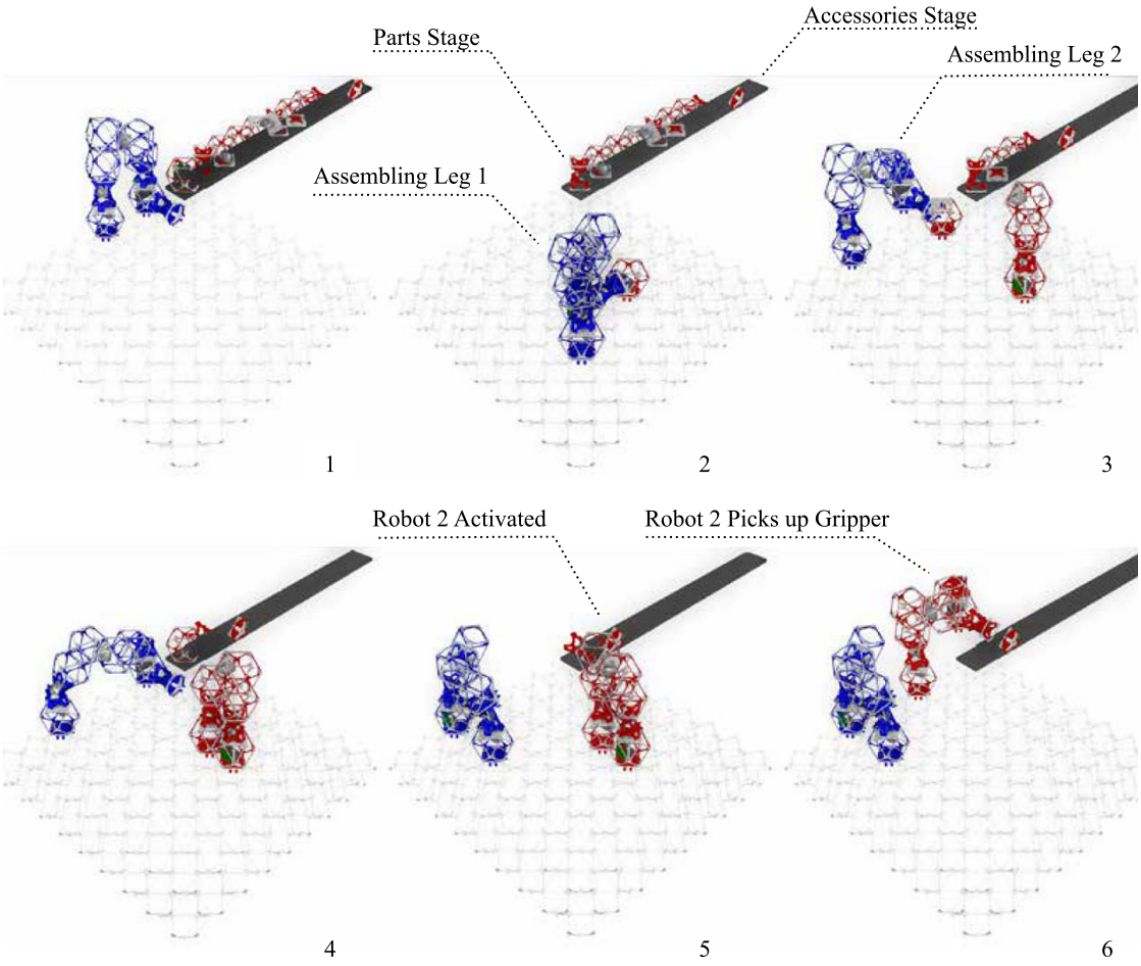

Figure 2.10: Self-replication process [Abd+22]

One of possible variations of assembler robots, designed to carry voxel cubes, and a process of self-replication from a set of modules is shown in Figure 2.10.

To perform self-replication, an initial "parent" robot uses its manipulators to pick and place voxels from a nearby pickup station. The parent robot builds a copy of itself—termed a "child" robot—by assembling voxels in a configuration that mirrors its own structure. The replication process is limited to voxels and certain key functional modules, including control units, power supplies, and actuated joints (called elbows). Components like wrists and grippers are added afterward through a procedure called "accessorizing," where the robot attaches these parts to the pre-assembled structure.

A critical constraint in the replication process is that the newly built robot must be free to move upon completion. Therefore, construction cannot proceed directly on the substrate base; instead, it extends out from an anchored platform, ensuring that once finished, the child robot can undock and function independently. The replication logic is programmed into a centralized control system, which assigns tasks, manages timing, and ensures robots do not collide. The control system guides each robot step-by-step in constructing a new robot, allowing for recursive reproduction over multiple generations.

This system's significance lies in its ability to scale construction tasks by increasing the number of robots through self-replication, rather than requiring the deployment of additional robots externally. As more robots are built, the workload is distributed, theoretically allowing exponential speed-up in large-scale assembly projects. The practical implementation uses inchworm-style locomotion, where robots move stepwise over the voxel lattice using coordinated grippers and joints, making them lightweight, simple, and adaptable to modular construction environments.

While this model is a proof-of-concept, it successfully demonstrates robotic self-replication using standardized parts and relatively low-complexity hardware. The modularity and discrete assembly approach also enhance the system's potential for error detection, robustness, and reconfiguration. Although scalability is limited by actuator strength and coordination complexity, the framework opens new avenues for autonomous robotic manufacturing systems that can grow their own workforce from a finite resource pool.

3 Research questions

This paper will address the following research questions:

1. How can automated model checking and other formal-verification techniques be used for verifying liveness (replication reliability), safety and other properties in self-replicating robotic systems(SRRS), and what domain-specific adaptations are required for practical implementation?
2. What systematic methodology can translate the behaviors and structural constraints of self-replicating robots into expressive temporal-logic models and specifications?
3. What limitations—computational, semantic, or modelling-related – does developed approach face, and how might these limitations be mitigated?
   Compare the finite state machine algorithm control with SMV counterexample-based control in simulation by time, flexibility

3.1 Analysis of model checking approaches in self-replicating systems domain

The first research question involves a comprehensive survey and critical evaluation of current state-of-the-art model checking and formal verification tools, such as SPIN, NuSMV, and PRISM, focusing on their applicability to modeling and verifying the unique dynamics of self-replicating robotic systems (SRRS).

This investigation includes a review of relevant literature and existing systems in self-replicating robotics and cellular automata to identify key operational characteristics and formalizable behaviors. It also involves identifying core properties relevant to SRRS, such as liveness (ensuring successful replication within time or resource constraints), safety (avoiding resource conflicts or hazardous outcomes), as well as broader system-level guarantees like termination, deadlock-freedom, and fault-tolerance. The study will further explore how these properties can be expressed in various formal languages, including Linear Temporal Logic (LTL), Computation Tree Logic (CTL), and

probabilistic extensions. This will require examining what adaptations or abstractions are needed to make these languages suitable for SRRS modeling.

Iterative experimentation with selected tools will help assess the feasibility of encoding and verifying these properties and may lead to the discovery of more complex or emergent behaviors to integrate in later stages.

3.2 Experimental checking

The second research question aims to define a systematic formal modeling and verification workflow tailored to SRRS. This involves developing a formal representation of self-replication processes, structural rules, environmental interactions, and resource usage.

It also includes establishing a methodology to map real-world or simulated robotic behaviors - such as perception, manipulation, and actuation- onto abstract states and transitions that are amenable to formal analysis. Based on this framework, appropriate model-checking tools and specification languages will be selected and justified through experimental evaluation within simulation environments.

A prototype or proof-of-concept implementation will be developed using a simplified SRRS scenario, such modular robotic replication, to demonstrate how this methodology captures essential properties and enables formal verification.

The ultimate goal is to bridge the gap between low-level robotic simulation and control systems and high-level formal models, ensuring that the designed behaviors are both physically feasible and logically justified.

3.3 Limitations and challenges

The third research question addresses the fundamental limitations that current formal methods and model-checking tools face when applied to the complex domain of self-replicating robotic systems. It includes analyzing computational constraints such as state-space explosion, undecidability in dynamic conditions, and potential inefficiencies in scaling existing verification tools.

The study will also examine semantic mismatches between physical or simulated robot behavior and abstract models, particularly in representing kinematic and dynamic aspects, sensor errors, and asynchronous or partially observable environments.

Additionally, the research will explore modeling limitations related to the representation of dynamic structure generation and adaptive behaviors.

To address these challenges, various mitigation strategies will be proposed, including modular verification, changes in system abstractions and hybrid techniques that combine formal models with simulation-based validation.

The outcome of this part may include recommendations for extending the selected verification toolsets, adapting implementation strategies, or designing new experimental setups specifically for the analysis of SRRS.

4 Methodology

4.1 Research tools

Self-replication research in robotics involves complex interactions between assembly processes, modular design, environmental constraints, and control systems. Robotic simulation environments offer a valuable sandbox to explore these dynamics before physical implementation. The ideal simulation platform must support modularity, kinematics, sensing, and interaction with discrete components.

A set of tools for simulation and control of robotic systems including SRRS can be seen in the Table 4.1

In order to simulate kinematic self-replicating systems and to be able to implement control functions that are also applicable to a physical system, the Gazebo simulator with ROS infrastructure seems to be an optimal choice.

Gazebo supports multiple physics engines, such as ODE, Bullet, and DART, allowing to model the dynamics of contact, friction, collisions, and gravity with a high degree of accuracy.

Modular robot design can be modeled through ROS's URDF (Unified Robot Description Format) and SDF (Simulation Description Format). These formats help to define robotic structures as combinations of interconnected parts and joints, closely mimicking the principles of modular self-replicating systems. Components can be described independently and reused across different robots or configurations. It support simulating a wide array of sensors, including cameras, LiDAR, depth sensors, IMUs, proximity sensors, and contact detectors.

Structured or unstructured environments for SRRS can be modeled in great detail, placing objects with specific physical properties, and simulating real-world challenges like terrain irregularities or sensor noise.

ROS provides a powerful middleware layer that manages communication between different components of the robot and the environment. Through ROS, we can implement and test high-level control algorithms, including perception pipelines, decision-making logic, and task planning. It supports distributed systems, so that multi-robot

Simulator	Physics fidelity	Modularity support	Custom control	Use case for SRRS
Gazebo+ ROS	High	Strong via URDF/SDF	ROS integration, plugins, Python/C++ nodes	Physical replication with mobile robots and manipulators
CoppeliaSim (V-REP)	High	Native modular modeling	Lua, Python /remote APIs	Detailed control of part interaction and modular assembly
PyBullet	Medium to High	Good (via URDF /SDF)	Python API (fast iteration)	Cube/voxel-based systems and contact-rich interactions
Webots	Medium	Moderate	C/C++ /Python controllers	Lightweight simulation of small-scale replication setups
Unity+ ML-Agents	Medium	High (custom prefabs)	C#,Python with ML-agents	Learning-based or perception-driven replication strategies
Isaac Sim (NVIDIA)	High	Strong (via Omniverse)	Python scripting, ROS2	High-fidelity simulation of complex autonomous robots
Golly NetLogo CA tools	None	Tile-based, symbolic	Rule-based/ scripting	Theoretical studies (e.g. von Neumann or Langton automata)

Table 4.1: Comparison of simulation environments for self-replicating robotic systems

scenarios (such as cooperative replication) can be easily simulated. ROS also provides tools for recording and analyzing data, tuning controllers, and implementing feedback loops.

Gazebo and ROS are open-source and highly extensible. Developers can create plugins for new sensors, actuators, or control strategies. This is particularly useful in self-replication research, where novel components or interfaces may need to be modeled. Additionally, the close integration between simulation and real-world systems means that code developed in Gazebo with ROS can often be deployed directly to real hardware with minimal changes.

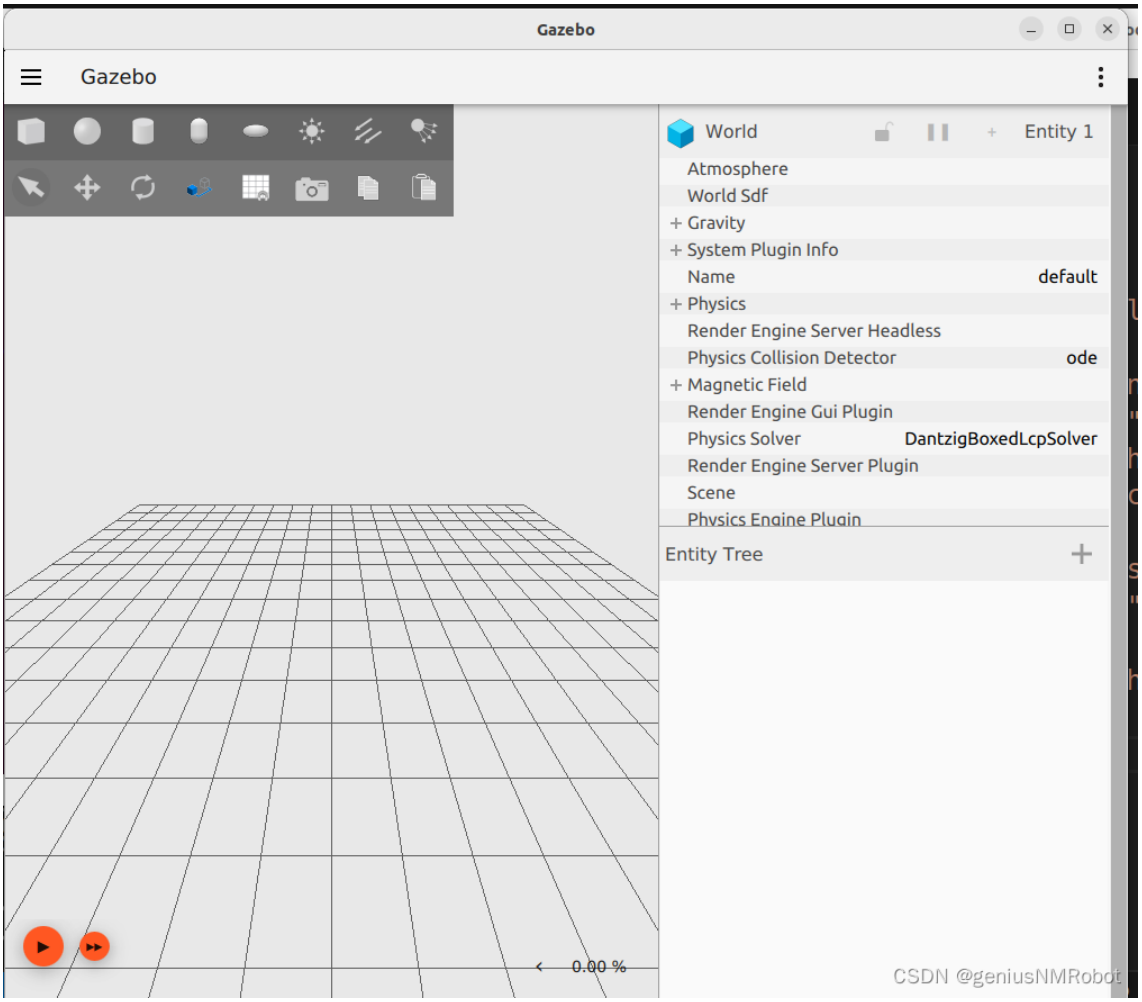

Figure 4.1: Gazebo Harmonic simulation interface

4.2 Proposed approach

Gazebo in combination with ROS can be used to create an interface between simulated robot behavior with formal verification tools, such as NuSMV. Formal methods allow researchers to rigorously verify properties of robotic systems—such as correctness, safety, liveness, and reachability —by constructing abstract models of their behaviors and validating them against logical specifications.

ROS-based systems define robotic structure and dynamics using URDF (Unified Robot Description Format) files and execute behavior through Python or C++ ROS nodes. These two sources—mechanical definitions and control logic—can be systematically translated into finite-state abstractions suitable for model checking with tools like

NuSMV.

The process typically involves analyzing the state transitions implied by the ROS node logic, especially within action sequences (e.g., part pickup, alignment , assembly), and discretizing them into states, inputs, and transitions. Each robot behavior or controller mode (e.g., "move", "grasp", "assemble") is represented as a finite state, while events and sensor conditions define transition rules. The mechanical configuration from URDF can define static properties or initial states in the system.

From this, a states diagram structure can be extracted, encoded in NuSMV's input language, and checked against temporal logic properties, described in CTL or LTL formulas. For instance, one could verify that a self-replicating robot always eventually reaches the assembly state (liveness) or that it never attempts to pick a module unless one is detected (safety).

Automating this translation—using Python scripts, that parse URDF and interpret ROS node behavior—is increasingly feasible, especially for systems designed with modular and rule-based architectures. Scripts or intermediate frameworks can analyze ROS bag files or logs to capture sequences of operations, and encode these as state transitions.

This capability enables hybrid verification workflows, where simulation is used to test behavior under dynamic conditions, and formal verification is applied to prove correctness across all possible system executions. In the context of self-replication, this allows verification of critical properties like "a complete replica is only activated after all modules are assembled" or "robots do not collide while cooperating on part assembly".

By linking Gazebo/ROS to NuSMV, can be created a powerful toolchain to not only test and observe behavior in simulation but also prove correctness and robustness in all logical conditions—an essential feature for deploying self -replicating systems in safety-critical or remote environments.

5 Implementation of model checking in SRRS

5.1 Overview

In this research we will consider a simplified model of a relative modular robot and corresponding environment, designed with a potential for self-replication and based on the design, and material-robot principle, developed by Jenett et al. and Abdel-Rahman et al.

In our model, as in the original work, robot Figure 2.9b operates by manipulating and connecting blocks Figure 2.9a of the same type as its own building blocks. These blocks can be considered as material for assembling different structures and are distributed in the environment in ordered or arbitrary manner. At any given time, robot in our model is capable of handling a single block, aligning it with the existing discrete environment and attaching it to some structure or new robot. Through repeated execution of this process, the robot can in principle assemble a replica of itself, thereby demonstrating the fundamental task of self-replication.

The robot is composed of a sequence of blocks of different functional types. The first and the last blocks of the sequence serve as end-effectors equipped with grippers, enabling the robot to interact with its environment. The intermediate blocks act as joints, which may provide rotations around different axis, for example changing yaw or pitch angles. Despite their functional diversity, all blocks are geometrically identical and take the form of cuboctahedral voxels, general view of it is shown in Figure 5.1. This uniform geometry facilitates a modular construction paradigm, where the same physical units can be rearranged to produce different structures and robot morphologies.

The robot operates within a structured environment defined as a lattice field that is geometrically compatible with the cuboctahedral voxel design. This lattice representation constrains both the robot and the individual building blocks to occupy discrete positions, ensuring that all spatial relations, movements, and connections can be described in terms of transitions between well-defined integer coordinates. Such discretization not only simplifies the modeling of robot kinematics and assembly opera-

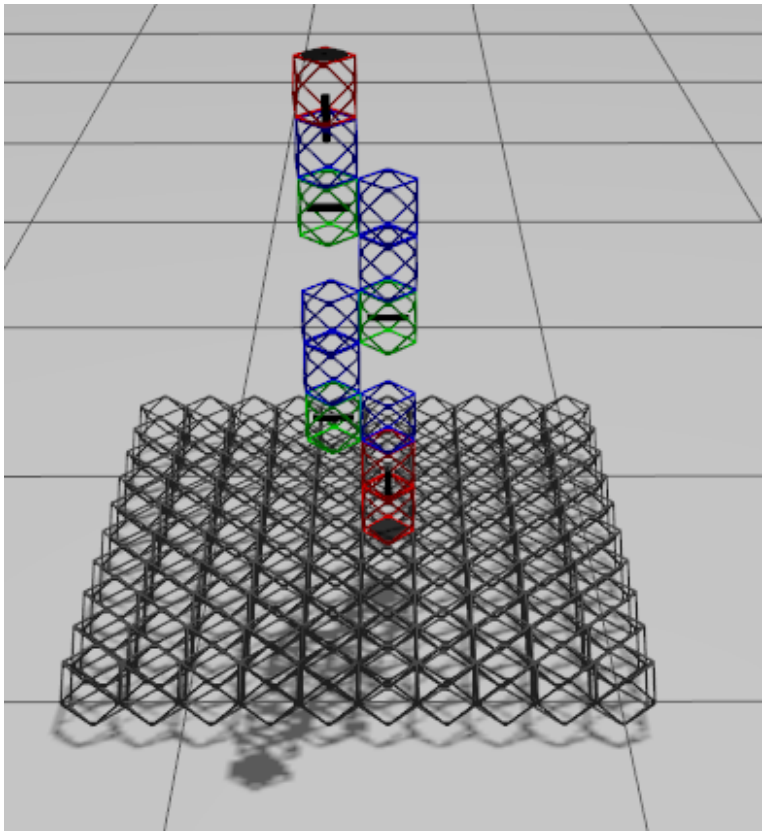

Figure 5.1: Robot structure in Gazebo simulation environment

tions but also provides a natural basis for formal verification, since the system state space becomes finite and directly available for exploration by model-checking techniques.

In this chapter, we focus on methods to formally analyze the behavior of such systems. Model checking provides a rigorous framework for verifying whether the system satisfies specific properties of interest. In particular, we investigate the use of symbolic model checker - NuSMV, to evaluate both safety and liveness conditions in the modular robot. Safety properties ensure that undesirable states, such as collisions or inconsistent connections, are never reached. Liveness properties, on the other hand, guarantee that the system is always capable of making progress, for example, that a new block can eventually be attached to a target or that a new robot can continue self-replication process.

Beyond simple verification, we also explore the potential of using NuSMV counterexample generation as a decision-making strategy. Counterexamples provided by the model checker correspond to concrete sequences of state transitions that lead to a vi-

olation of a property. Interpreted constructively, such sequences can serve as action plans or decision sequences for the robot, indicating feasible operations that allow it to achieve its objectives. This approach establishes a bridge between formal verification and automated planning and can increase the reliability and effectiveness of the system, as well as add flexibility to decision-making in complex and dynamic conditions.

TODO: Add individual block geometry figure

5.2 Analysis of the system

The self-replicating system under consideration 5.1 for the general configuration of a robot can be classified as active system with hierarchical type of replication and capability for direct and indirect replications.

It is hierarchical because it can produce robots capable of self-replication as well as robots or mechanical structures lacking of replication capability. It is active since it is engaged in constructing child robots and

The system can be homogeneous or heterogeneous based on initial conditions and programmed behavior. As described in 2.3.2 homogeneous type of SRRS consist of robotically identical units, whereas heterogeneous enables generation of multiple robot types, each potential for specialization depending on environmental conditions.

For the sake of simplicity, we will assume the system is homogeneous but will analyze effect of different types or configuration of robots in different environments on the properties of a system.

Using model proposed by Lee and Chirikjian in [LC07] and described in 2.3.3 our self-replicating system can be formally represented as Formula 5.1.

$$(R', M', E) = \Phi(R, M, E, t) \tag{5.1}$$

Where:
M – multiset (set with repetitions) of available parts of a robot parts located on the base to be used for building replicas robots by an initial system.
R – a multiset of initial robots to be reproduced. In our case one robot with predefined configuration. This includes robot structure in form of sequence of blocks, geometry, kinematic limitations and robot control system – software and hardware parts.
E – is a multiset of environmental structures and/or elements involved in the replication process (but not replicated). In our experiment is represented by base and optional

obstacles.
R', M' – the original and build entities, the remaining and altered material respectively.
Φ – the replication function of time t.

This model is a special case of a Formula 2.2 for completely structured environment. Because every element of the environment is either fixed at a precise location, or described in integer coordinates and there will be no modification or change in any structures during the self-replication process.

This formulation enables modeling dynamic replication scenarios over time and accounts for state changes in materials and set of robots.

To sum up the description above we have a material-robot system, where the environment material unit introduced by the robot part, modular robot structure with small amount of block types, and completely structured discrete environment.

Such systems can be described as transition systems and is therefore suitable for model checking. It can be checked for specific requirements, written in form of temporal logic.

As a central process of the system we will consider an act of self-replication – an assembly of a replica of itself by the initial robot from a set of parts located on a base. This process shown in state chart on Figure 5.2.

In this self-replication process we can derive following actions:

1. Connection and disconnection of the end-effector to and from the block or base.
2. Localization of the part, or receiving of part coordinates from the sensors. This action is a part of "Checking assembly state" and "Searching block in local area" in state chart.
3. Movement of the end-effector from one point to another, with or without connected parts.

Implementation of connection and disconnection actions is platform dependent be represented in a control system as boolean state. Implementation of parts localization actions also depends on specific sensors configuration and similarly could be a represented as a boolean array. Movement action depends mainly on the robot blocks configuration, blocks parameters, environmental limitations, initial and target points. As a result, model checking of generalized movement has a potential to substantially improve the overall system self-replication process and adapt it for different environment conditions. In the following sections we will model robot, translate self-replication process in transition system and check following properties using NuSMV:

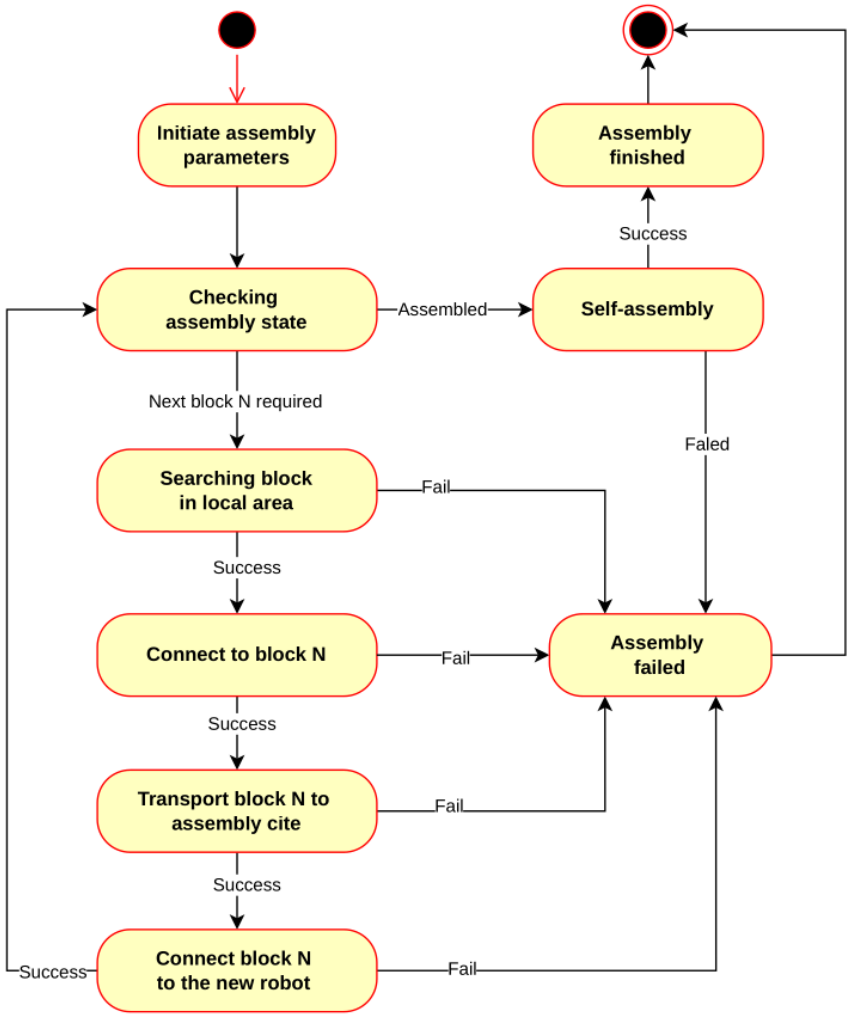

Figure 5.2: State chart of self-replication assembly process

- Check local reachability of some point or a set of points (region) by the robot end-effector.
- Generate a command sequence for a robot to move the end-effector to target point, based on counterexamples.
- Research global reachability of some point or region by robot end-effector using the walking form of movements.
- Generate a path to a point using walking based on counterexamples.

- Analyze set of robot configurations – check existing configuration or find relevant configurations of blocks based on counterexamples.

plan experiments: Criteria: success rate, average time,
1. Test with predefined assembly process (search, move, repeat) with random distribution of the parts
2. Test with predefined assembly process (...) with , criteria and analysis

TODO: 2. Correct diagram to SYSML block connection diagram. remove transition system

5.3 Model checking based control in local space

5.3.1 Overview

Input: Initial position, target position, set of obstacles Robot structure (xacro) -> Generate NuSMV -> check model, get counterexample -> derive path -> compare with shortest path algorithm (implement Deikstra/ A*)

Evaluation: comparison, Validation: assembly simulation in Gazebo

5.3.2 Robot NuSMV model

The robot structure as it shown in Figure 5.1 is represented as a modular composition of three distinct block types: a fixed base block, blocks with a yaw degree of freedom, and blocks with a pitch degree of freedom. Figure 5.3 illustrates the hierarchical arrangement of these blocks and the flow of coordinate and orientation data between them. This modular abstraction allows the robot structure to be described in a uniform and scalable manner, independent of the number of blocks.

As the lowest level of a system, as it shown in Figure 5.3 we define a block. Each subsequent block introduces a transformation of the block end position according to its type. Each block has a certain type, in this work i consider three types of blocks:
The $block\_base$ defines the fixed starting point of the robot. It receives as input the global origin coordinates (x_orig, y_orig, z_orig) and the initial orientation parameters $(yaw\_orig, pitch\_orig)$ Its role is to provide a reference frame for all subsequent blocks. The base block outputs the coordinates (x_end, y_end, z_end) and orientation values that are propagated forward to the next block in the sequence.

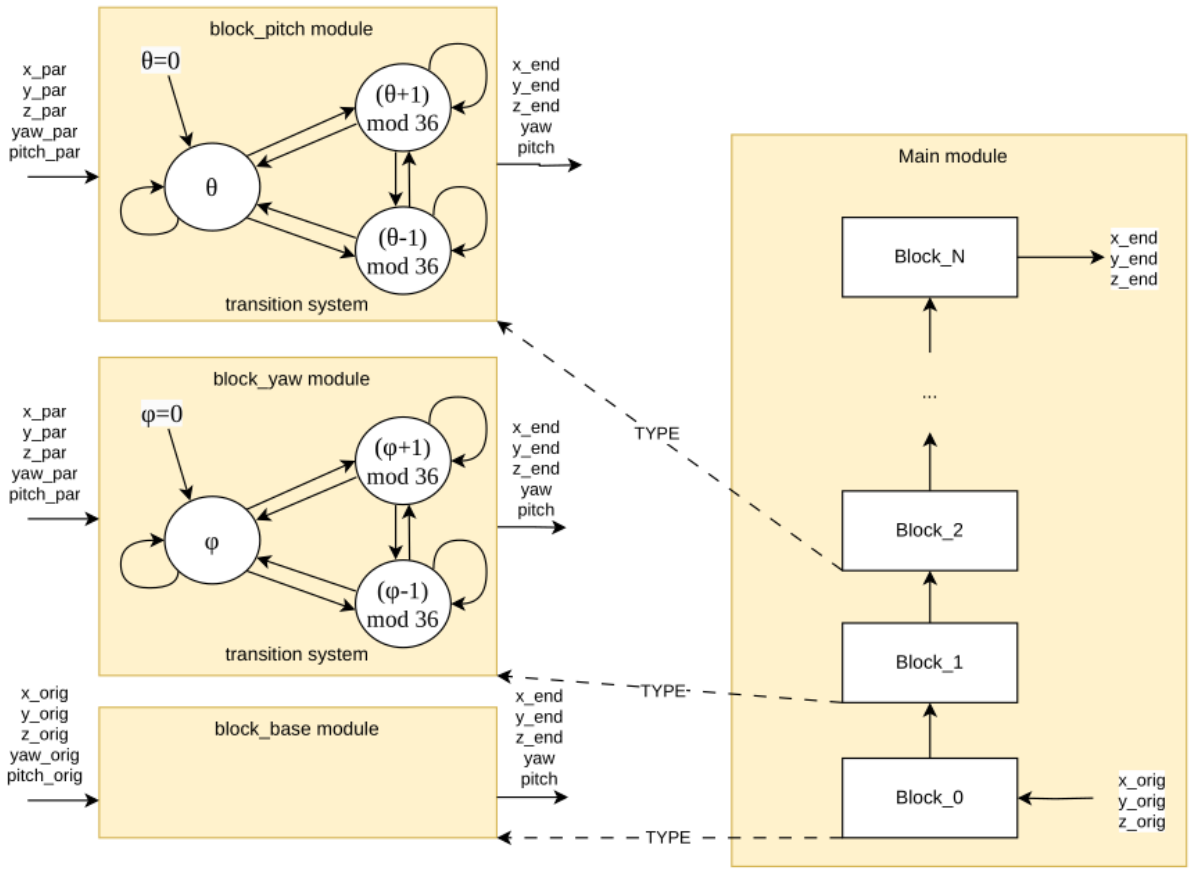

Figure 5.3: Robot configuration NuSMV model

Block types block_yaw realise rotation around local axis Z, that oriented along the parent block The $block\_yaw$ represents a joint capable of rotating around the vertical axis. Internally, the block maintains a discrete variable ϕ, which denotes the yaw angle.

As illustrated in the Figure 5.4, ϕ can evolve in steps of +1 or -1, modulo 36, corresponding to 10-degree discretization over the full circle of 360°. Thus, the block admits cyclic transitions that allow continuous rotation in both clockwise and counterclockwise directions. The block takes as input the parent coordinates (x_par, y_par, z_par) and orientation $(yaw\_par, pitch\_par)$, applies the yaw rotation, and produces updated endpoint coordinates.

$$\begin{cases} \phi \to \phi \\ \phi \to (\phi + 1)\ mod\ 36 \\ \phi \to (\phi - 1)\ mod\ 36 \end{cases} \tag{5.2}$$

Block types block_pitch realise rotation around local axis Y, that is perpendicular to Z often the parent block. Similarly, the $block\_pitch$ encodes a joint that rotates around the lateral axis of the preceding link. Its discrete state variable θ:

$$\begin{cases} \theta \to \theta \\ \theta \to (\theta + 1)\ mod\ 36 \\ \theta \to (\theta - 1)\ mod\ 36 \end{cases} \tag{5.3}$$

In combination, these transitions capture the continuous ability to rotate up or down by discrete steps.

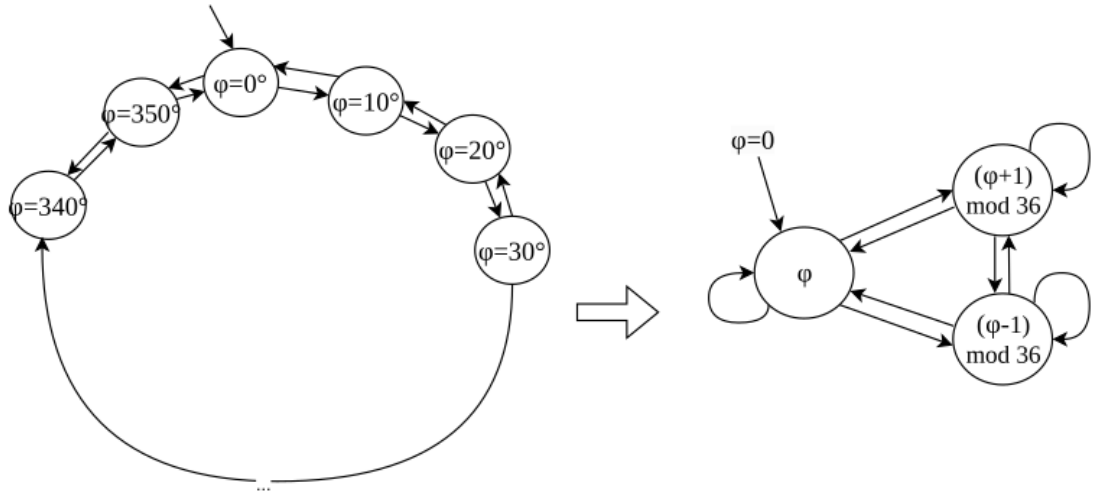

Figure 5.4: Robot block transition system for angle ϕ

At the system level, the robot is built as a chain of such blocks. After the base block, one may attach any sequence of yaw and pitch blocks in arbitrary order, depending on the desired manipulator structure. The right-hand side of Figure 5.3 shows an example with N chained blocks, where Block 0 connects directly to the base and Block N defines the robot's end-effector. The outputs of the last block are new coordinates (x_end, y_end, z_end) and updated orientation values. Each block is parameterized by the output of its predecessor, creating a cascading dependency structure.

Thus, on the NuSMV model description level we have:

- main block module - that describes a robot structure as a sequence of block types as variables in VAR section.
- block_base module - describing initial block of a sequence and locating on a ground plane.

```
MODULE block_yaw(px, py, pz,
                 xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz,
                 L, initYaw)
VAR
  yaw : 0..35;
  cy : cos(yaw);
  sy : sin(yaw);

DEFINE
  SCALE := 100;

  cos_yaw := cy.val;    -- [-100..100]
  sin_yaw := sy.val;

  xhx2 := (xhx * cos_yaw - xhy * sin_yaw) / SCALE;
  xhy2 := (xhx * sin_yaw + xhy * cos_yaw) / SCALE;
  xhz2 := xhz;

  yhx2 := (yhx * cos_yaw - yhy * sin_yaw) / SCALE;
  yhy2 := (yhx * sin_yaw + yhy * cos_yaw) / SCALE;
  yhz2 := yhz;

  zhx2 := zhx;
  zhy2 := zhy;
  zhz2 := zhz;

  px2 := px + (zhx2 * L) / SCALE;
  py2 := py + (zhy2 * L) / SCALE;
  pz2 := pz + (zhz2 * L) / SCALE;


ASSIGN
  init(yaw) := initYaw;
  next(yaw) := { yaw, (yaw+1) mod 36, (yaw+35) mod 36 };
```

Listing 5.1: NuSMV code for block_yaw module

- block_yaw module - describing rotation around the parent Z axis (along the block) and changing the yaw angle.
- block_pitch module - describing rotation around the parent Y axis (perpendicular to the block) and changing the pitch angle.

In NuSMV block rotating around Z represented by module block_yaw, and shown in Listing 5.1

MuSMV modules block_base, block_pitch sin, and cos NuSMV modules shown in Appendix A. This design supports a natural inductive description of the robot in NuSMV: the same block type can be instantiated repeatedly, with only the input-output connections altered.

The modularity of the design has two important advantages.
First, it allows scalability, since additional joints can be incorporated simply by adding

```
ASSIGN
    ...
    init(yaw)    := 0;
    next(yaw)    := {
                     yaw,
                     (yaw+1) mod 36,
                     (yaw+35) mod 36
                     };
```

Listing 5.2: Assign part of code for block transitions

new instances of yaw or pitch modules.
Second, it supports module based model checking, because each block is described as a transition system with a finite number of states and transitions. The global state space of the robot is then obtained by synchronous product of the component transition systems, which can be directly analyzed using LTL or CTL specifications in NuSMV.

The dynamics of each yaw or pitch block can be represented as a finite-state transition system. Figure 5.4 illustrates this equivalence explicitly. On the left-hand side, the system is shown as a ring of discrete states corresponding to the possible angle values

$$\phi = 0^\circ, 10^\circ, \ldots, 350^\circ$$

From each state, there are transitions to the neighboring states at $\phi + 10^\circ$ and $\phi - 10^\circ$ thus modeling incremental rotations in either direction. Since the variable is defined modulo 36, the state $\phi = 350^\circ$ transitions back to $\phi = 0^\circ$, completing the cycle. The result is a finite cyclic transition system with 36 nodes, which represents the discretized continuous rotation of the block.

On the right-hand side of Figure 5.4, this same behavior is captured more compactly. Here, the internal state variable ϕ is updated according to transition rules:

$$\phi = \phi \quad \vee \quad \phi = (\phi + 1)\ mod\ 36 \ \vee \phi = (\phi - 1)\ mod\ 36$$

This concise representation preserves the semantics of the explicit ring but makes the module description more compact and scalable. When combined with coordinate update formulas, it fully captures the contribution of the block to the robot's kinematics.

This transitions and angle changing for each block reflected by the following part of assign block, shown in Listing 5.2.

For each block in a sequence of robot structure the initial value is assigned, and next state can be chosen nondeterministically from three possible transitions.

```
DEFINE
...
error  := 1;
checkX := 10;
checkY := 15;
checkZ := 25;
endX_inside_limits :=(endX <=(checkX+error) & endX >=(checkX-error));
endY_inside_limits :=(endY <=(checkY+error) & endY >=(checkY-error));
endZ_inside_limits :=(endZ <=(checkZ+error) & endZ >=(checkZ-error));
```

Listing 5.3: End effector and target point coincidence condition

The full NuSMV model for modules: block_base, block_yaw, block_pitch and main presented in Appendix A.

Point reachability checking specifications:

In order to verify whether the robot is capable of reaching a desired target point with its free end-effector, in three-dimensional space, we formalize the reachability check directly within the NuSMV model.

Since we can use only countable number of states and have to represent angles as integers, each transformation of angles to linear coordinates is not precise. Correspondingly the distance between actual and target points is between zero and linear delta calculated based on descritization value for angle.

The relevant fragment of code is shown in Listing 5.3.

Here, the variables endX, endY, and endZ represent the Cartesian coordinates TODO: add clarification about CS translation and of the robot end-effector, which are computed elsewhere in the model based on the joint angles and link lengths. The parameters checkX, checkY, and checkZ denote the coordinates of a target point that we wish to test for reachability. Since the system operates with discretized values, a tolerance margin error is introduced. For each spatial dimension, Boolean formulas $endX\_inside\_limits$, $endY\_inside\_limits$, and $endZ\_inside\_limits$ check whether the end-effector coordinate lies within a small interval around the corresponding target value, effectively defining a cube of side length 2*error centered at the target.
The formal property is expressed in Linear Temoporal Logic (LTL) in Listing 5.4.

states that on every execution path, it will always (G) eventually (F) be the case that the robot's end-effector is not within the specified tolerance region around the target point. In other words, the property enforces that the end-effector infinitely often leaves

```
LTLSPEC
G !(endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.4: Reachability condition in LTL specification

```
CTLSPEC
AG !(endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.5: Reachability condition in CTL specification

the target zone, meaning the robot cannot remain in that region permanently.
The formal property is expressed in Computation Tree Logic (CTL) in Listing 5.5.

The specification:

$$AG\neg(endX\_inside\_limits \wedge endY\_inside\_limits \wedge endZ\_inside\_limits)$$

is interpreted as: "On all computation paths (A), in all states globally (G), it is never the case that the end-effector simultaneously satisfies all three coordinate bounds." In other words, if this property holds, the model checker proves that the robot can never reach the point (10,15,25) (10,15,25) within the given tolerance.
Conversely, if the property is violated, NuSMV produces a counterexample trace that demonstrates a specific sequence of joint configurations leading the end -effector into the defined target region. This counterexample can be directly interpreted as a feasible motion plan achieving the desired reachability.

Region reachability checking specifications:

We can check reachability of some 3D region, in our case a box by defining the variables checkX, checkY and checkZ as FROZENVAR. FROZENVAR declares parameters that are chosen nondeterministically at starting time and then never change along a run for an entire execution (trace). The Listing 5.6 represents an example of a region:
$x = [-10, 10], y = [-10, 10], z = [0, 20]$.

```
FROZENVAR
    checkX : -10..10;
    checkY : -10..10;
    checkZ : 0..20;
```

Listing 5.6: Region definition for a reachability checking

```
CTLSPEC
   EF (endX_inside_limits & endY_inside_limits & endZ_inside_limits )
```

Listing 5.7: Reachability condition of a region with CTL specification

Changing in the specification for the region checking using CTL logic shown in Listing 5.7. *EF* means "there exists a path along which eventually holds following condition". With the frozen variable region, the formula states: from every initial state (for every admissible target chosen at t=0), there exists a sequence of moves that eventually places the end-effector inside the region.

Because NuSMV checks CTL properties over all initial states , this specification proves reachability for every target in the declared ranges (universal over targets, existential over paths).

These specifications therefore provides a rigorous, formally verifiable mechanism to test spatial reachability of robotic manipulators in discrete state-space models. It can be generalized by altering the checkX, checkY, and checkZ parameters, and by varying the tolerance error, to systematically analyze the reachable workspace of the system.

5.3.3 Experimental setup

TODO: plan experiments: 1. Test with predefined assembly process (search, move, repeat) with random distribution of the parts
2. Test with predefined assembly process (...) with, criteria and analysis Criteria: success rate, average time, disagreement with control alg

Experiment 1:
Subject: local assembly with shortest path control algorighm
Setup: random distribution of blocks and obstacles in 3D (for 0-15 obstacles) Measure values: average assembly time, average success rate

Experiment 2:
Subject: local assembly with path control algorighm and runtime verification
Setup: random distribution of blocks and obstacles in 3D (for 0-15 obstacles) Measure values: average assembly time, average success rate

Experiment 3:
Subject: local assembly with runtime verification and counterexample based control algorighm

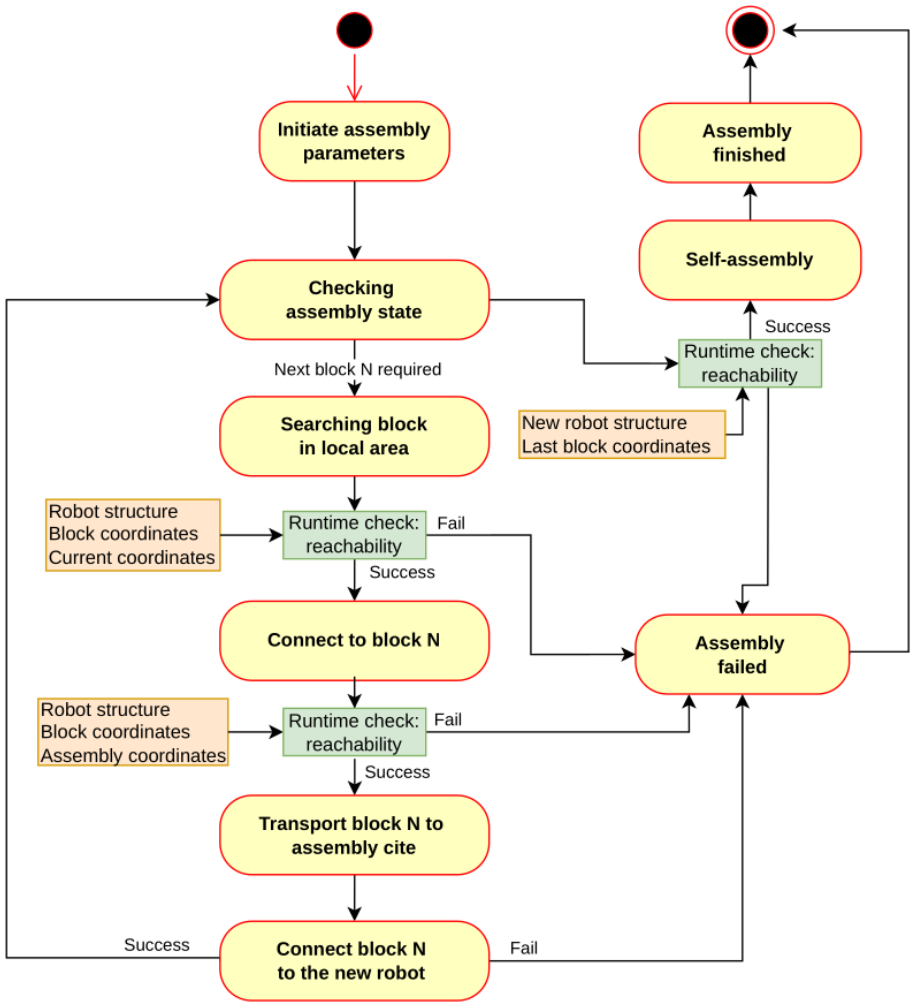

Figure 5.5: Self-replication process with potential model checking stages

Setup: random distribution of blocks and obstacles in 3D (for 0-15 obstacles) Measure values: average assembly time, average success rate

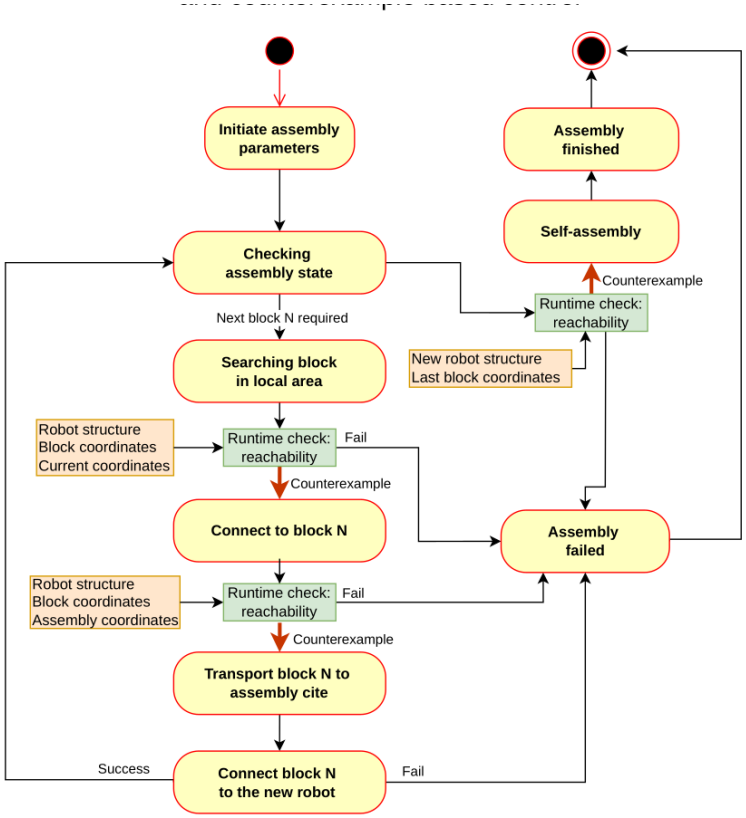

Figure 5.6: Local assembly process without model checking

5.4 Robot configuration model checking

5.4.1 Overview

In this part we will check the model of the robot of the certain configuration for the reachability property introduced in

Block types -> NuSMV block modules structure -> NuSMV template
Border conditions: - Base blocks on the end - initial block position - connect only from upper side

python script -> Generate NuSMV -> check reachability border

Criteria: reachability of the random distributed blocks, given base block position Validation: optimal structure -> generate xacro, check assembly

5.4.2 Searching for relevant robot configurations

Individual robot configuration checking

In order to systematically evaluate which robot structures are capable of reaching a target coordinates, implemented an automated configuration checking procedure that integrates model generation, model checking with NuSMV, and result filtering. The overall workflow is illustrated in Figure 5.7.

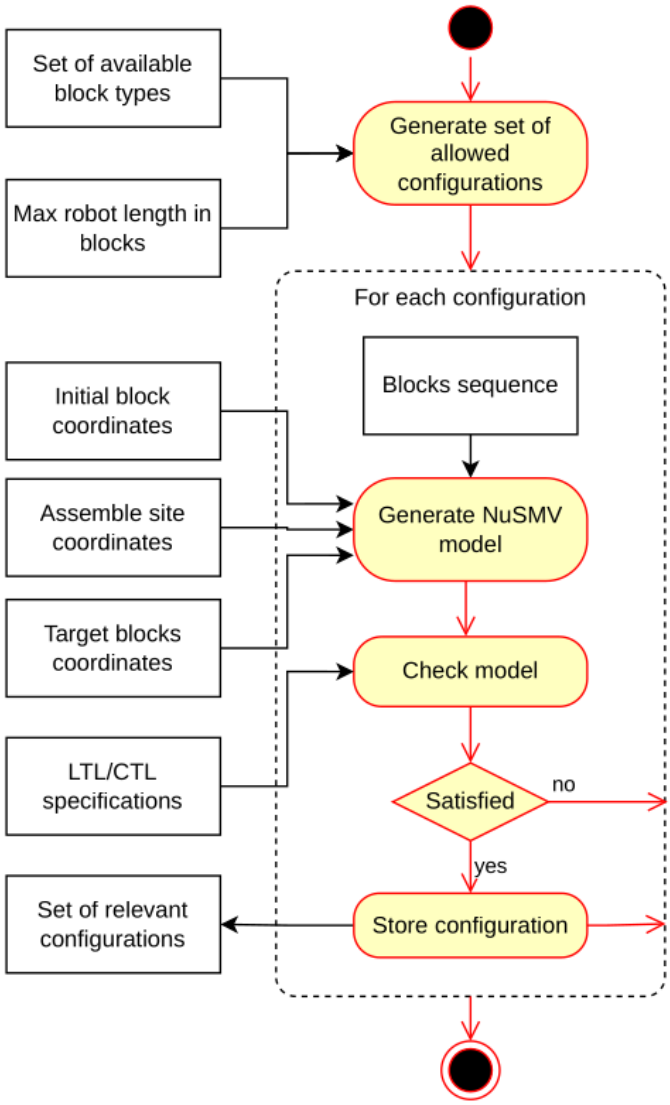

Figure 5.7: Activity diagram of searching for relevant configurations

The process of checking robot configurations is performed by integrating Python - based model generation with the NuSMV model checker. The workflow begins with two primary inputs: the set of available block types, such as base, yaw-rotating, and pitch-rotating blocks, and the maximum robot length expressed as the number of blocks. These inputs are used to generate the complete set of allowed configurations of length N, ensuring that every possible arrangement of building blocks within the specified limit is considered. Each configuration is represented as an ordered sequence of blocks, starting with the mandatory base block and followed by a selection of yaw and pitch blocks. Once the set of configurations is created, the process iterates over each configuration individually.
The block sequence is extracted and combined with:

- coordinates of an initial block of a robot (x_{orig},y_{orig},z_{orig})
- assembly site coordinates relative to a robot(x_{asm},y_{asm},z_{asm})
- target block coordinates relative to a robot: [Block(x_1,y_1,z_1), Block2(x_2,y_2,z_2), ...]

This information forms the basis for generating a concrete NuSMV model through a Python script. The script instantiates the appropriate modules corresponding to each block type, establishes the connections between their input and output variables, and integrates the positional and orientation parameters across the robot chain. In addition , the script embeds the temporal logic specifications provided as LTL or CTL formulas. These specifications define the verification goals, such as whether the end-effector can reach a given target coordinates and avoids certain unsafe areas.

The generated model is then passed to NuSMV, which executes symbolic model checking to evaluate whether the specifications are satisfied. The model checker systematically explores the entire state space of the configuration, taking into account the discrete transitions of yaw and pitch blocks as well as the coordinate propagation along the chain.
If the specification is not satisfied, the algorithm discards the configuration and proceeds to the next candidate in the sequence.
If the specification is satisfied, the configuration is stored in a set of relevant configurations, which represents the final output of the process.

Implementation in Python

The implementation of the workflow described in Figure 5.7 is presented in three-class Python design. The architecture of the design corresponds directly to the class diagram

shown on the Figure 5.8, where the class *Configuration_checking* orchestrates the process, the class *Robot* assembles a sequence of blocks, and the class *Block* encapsulates the description of an individual block.

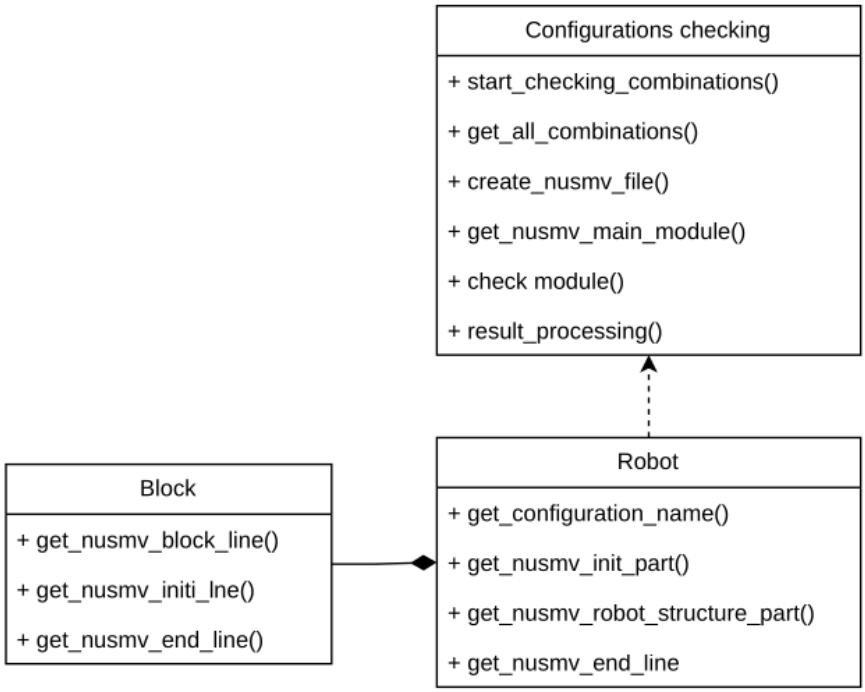

Figure 5.8: Class diagram of searching for relevant configurations

A *Block* class is the minimal self-contained representation of a kinematic segment in the robot block sequence. Each block is identified by an integer number and a type such as “base”, “yaw”, or “pitch”, and it stores the index of its parent block in order to define the topology and orientation. Geometric information is reduced to a single parameter length, while all pose and orientation information is described later through the NuSMV model. The block provides three critical outputs in the form of NuSMV code fragments: a structure line that instantiates the variable with appropriate block module and wires it to its parent, an initialisation line that sets link length and default joint angle values, and, for the final block, assignment lines that expose the end-effector coordinates. For non-base blocks, angle variable names are generated systematically (e.g., yaw2, pitch3), guaranteeing uniqueness and readability. In this way, each block fully describes its structural role without knowledge of the entire robot, making the class simple and robust.

The Robot class acts as a container and assembler for blocks. Constructed from a list

of block types, it automatically creates a chain of Block instances , numbers them, and links each to its parent. It maintains the list of blocks as well as a handle to the final block, which represents the robot's end-effector. The Robot then concatenates the fragments provided by its blocks into complete sections suitable for insertion into a NuSMV model. It can return the structure of the chain as a sequence of module instantiations, the initialisation of link lengths and angles, and the assignment of global end-effector coordinates. Additionally, it encodes the configuration in a readable name such as "base_yaw_pitch_yaw", which is used both in logging and file management. The Robot therefore translates an abstract sequence of types into a structured collection of blocks, and from there into concrete model code that describes the robot in full.

The third class, Configuration_checking, governs the entire process. Its role is to generate and test all possible configurations of a specified chain length composed of a given set of block types. The first block is always fixed to a base, and the remaining positions are filled with every possible combination of the admissible types. This results in an exponential search space, for example two types and four positions yielding sixteen unique robots. Configuration_checking takes as additional input a target point in space and stores both this point and the error tolerance around it, against which reachability is later checked. For each generated morphology, the class constructs a Robot, retrieves its NuSMV code, and splices it into a template file. The method create_nusmv_from_template searches for the region between "MODULE main" and a predefined end marker, replaces that region with a new main module generated for the current robot , and leaves the rest of the file untouched. This ensures that all library code, specifications, and fairness constraints in the template remain intact . The new module contains the structure lines from the robot, the initialisation of lengths and angles, the fixed base pose and basis vectors, the end -effector definitions, and the error-band predicates that compare the end coordinates to the desired target. This layering is explicit in the class diagram Figure5.8: Configuration_checking depends on Robot, which is composed of Block instances.

Instead of hard-coding a complete NuSMV model, the system rewrites the MAIN module fragment inside a reusable template file. This makes the pipeline resilient to changes in property specifications or library modules—the template remains the source of truth, while the classes inject only the structure and constants.

The overall workflow combines automatic model generation, model checking, and storing of satisfiable robot configurations. By using a Python script to handle tasks of configuration expansion and NuSMV code generation, the approach ensures scalability and reduces the potential errors. The inclusion of temporal logic specifications provides a basis for determining whether a configuration is functionally capable of achieving the task requirements. The final set of relevant configurations captures only

those robot designs that satisfy the given conditions, providing a reliable foundation for further analysis or deployment in robotic applications.

Aggregated robot configuration checking in NuSMV

5.5 Model checking based control in global space

5.5.1 Overview

Input: Initial position, target position, set of obstacles Robot structure (xacro) -> Define step algorithm -> optimal step?-> Generate NuSMV -> check model, get counterexample -> derive path -> compare with shortest path algorithm (implement Deikstra/ A* in 2D)

Evaluation: comparithon with shortest path algorithms Validation: assembly simulation in global space in Gazebo

5.5.2 Evaluation

5.6 General System Model checking

5.6.1 Overview

Input: Initial position, target position, set of obstacles Robot structure (xacro) -> Define step algorithm -> optimal step?-> Generate NuSMV -> check model, get counterexample -> derive path -> compare with shortest path algorithm (implement Deikstra/ A* in 2D)

Evaluation: comparithon with shortest path algorithms Validation: assembly simulation in global space in Gazebo

5.6.2 Evaluation

5.7 Collective behavior model checking

5.7.1 Overview

6 Validation and simulation

7 Evaluation

Bibliography

[Pen58] L. S. Penrose. “Mechanics of Self-Reproduction”. In: *Annals of Human Genetics* 23.1 (Nov. 1958), pp. 59–72. DOI: 10.1111/j.1469-1809.1958.tb01442.x (cit. on p. 19).

[NB62] John von Neumann and Arthur W. Burks. *Theory of Self-Reproducing Automata*. Edited and completed by Arthur W. Burks. Urbana, IL: University of Illinois Press, 1962 (cit. on p. 21).

[Azi+96] Adnan Aziz et al. “Verifying Continuous Time Markov Chains”. In: *Proceedings of the 8th International Conference on Computer Aided Verification (CAV)*. Ed. by Rajeev Alur and Thomas A. Henzinger. Vol. 1102. Lecture Notes in Computer Science. New Brunswick, NJ: Springer, 1996, pp. 269–276 (cit. on p. 11).

[Sip98] Moshe Sipper. “Fifty Years of Research on Self-Replication: An Overview”. In: *Artificial Life* 4.3 (1998), pp. 237–257. DOI: 10.1162/106454698568576 (cit. on p. 15).

[SZC02] Jackrit Suthakorn, Yu Zhou, and Gregory S. Chirikjian. “Self-Replicating Robots for Space Utilization”. In: (2002) (cit. on p. 19).

[YS02] Håkan L. S. Younes and Reid G. Simmons. “Probabilistic Verification of Discrete-Event Systems Using Acceptance Sampling”. In: *Computer Aided Verification (CAV ’02)*. Vol. 2404. Lecture Notes in Computer Science. Copenhagen, Denmark: Springer, 2002, pp. 223–236. DOI: 10.1007/3-540-45657-0_19 (cit. on pp. 10, 11).

[SKC03] Jackrit Suthakorn, Yong T. Kwon, and Gregory S. Chirikjian. “A Semi-Autonomous Replicating Robotic System”. In: *Proceedings of the 2003 IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA)*. IEEE International Symposium on Computational Intelligence in Robotics and Automation, Vol. 2. Kobe, Japan: IEEE, July 2003, pp. 776–781. DOI: 10.1109/CIRA.2003.1222279 (cit. on p. 20).

[Mic04] Mark Ryan Michael Huth. *Logic in computer science Modelling and Reasoning about Systems*. 2nd. Cambridge, New York, Melbourne, Madrid, Cape Town, Singapore, São Paulo: Cambridge University Press, 2004 (cit. on pp. 3–5, 7, 8, 10).

[Zyk+05] Victor Zykov et al. "Self-reproducing machines". In: *Nature* 435.7038 (2005). Brief Communication, p. 163. DOI: 10.1038/435163a (cit. on pp. 27, 28).

[You06] Håkan L. S. Younes. "Statistical Probabilistic Model Checking with a Focus on Time-Bounded Properties". In: *Information and Computation* 204.9 (Sept. 2006), pp. 1368–1409. DOI: 10.1016/j.ic.2006.03.002 (cit. on p. 10).

[EML+07] Steven Eno, Lauren Mace, Jianyi Liu, et al. "Robotic Self-Replication in a Structured Environment without Computer Control". In: *Proceedings of the 2007 IEEE International Conference on Robotics and Automation (ICRA)*. 2007 (cit. on p. 22).

[LC07] Kiju Lee and Gregory S. Chirikjian. "Robotic Self-Replication: A Descriptive Framework and a Physical Demonstration from Low-Complexity Parts". In: *IEEE Robotics & Automation Magazine* 14.4 (2007), pp. 34–43. DOI: 10.1109/MRA.2007.910 (cit. on pp. 15, 16, 19, 22–27, 41).

[KNP11] Marta Kwiatkowska, Gethin Norman, and David Parker. "PRISM 4.0: Verification of Probabilistic Real-Time Systems". In: *Computer Aided Verification (CAV 2011)*. Ed. by Ganesh Gopalakrishnan and Shaz Qadeer. Vol. 6806. Lecture Notes in Computer Science. Springer, 2011, pp. 585–591. DOI: 10.1007/978-3-642-22110-1_47 (cit. on p. 11).

[HSZ14] Timothy L. Hinrichs, A. Prasad Sistla, and Lenore D. Zuck. "Model Check What You Can, Runtime Verify the Rest". In: *HOWARD-60: A Festschrift on the Occasion of Howard Barringer's 60th Birthday*. Ed. by Andrei Voronkov and Margarita V. Korovina. Vol. 42. EPiC Series in Computing. EasyChair, 2014, pp. 234–244. DOI: 10.29007/slnn. URL: https://easychair.org/publications/paper/tq7 (cit. on p. 12).

[LST16] Axel Legay, Zacharias R. M. Sedwards, and Louis-Marie Traonouez. "Plasma Lab: A Modular Statistical Model Checking Platform". In: *ISoLA 2016 – Leveraging Applications of Formal Methods, Verification and Validation*. Ed. by Tiziana Margaria and Bernhard Steffen. Vol. 9952. Lecture Notes in Computer Science. Springer, 2016, pp. 77–93. DOI: 10.1007/978-3-319-47166-2_6 (cit. on p. 11).

[SL16] Valentin Goranko Stéphane Demri and Martin Lange. *Temporal Logics in Computer Science Finite-State Systems*. 2nd. University Printing House, Cambridge CB2 8BS, United Kingdom: Cambridge University Press, 2016 (cit. on pp. 8, 9).

[Deh+17] Christian Dehnert et al. "A Storm is Coming: A Modern Probabilistic Model Checker". In: *Computer Aided Verification (CAV 2017)*. Ed. by Isil Dillig and Jens Palsberg. Vol. 10426. Lecture Notes in Computer Science. Springer, 2017, pp. 592–600. DOI: 10.1007/978-3-319-63390-9_31 (cit. on p. 11).

[Jen+19] Benjamin Jenett et al. “Material–Robot System for Assembly of Discrete Cellular Structures”. In: *IEEE Robotics and Automation Letters* 4.4 (2019), pp. 4019–4026. DOI: 10.1109/LRA.2019.2930486 (cit. on pp. 29, 39).

[Tan+20] Ning Tan et al. “A Framework for Taxonomy and Evaluation of Self-Reconfigurable Robotic Systems”. In: *IEEE Access* 8 (2020), pp. 13969–13986. DOI: 10.1109/ACCESS.2020.2965327 (cit. on p. 15).

[JS21] Andrew Jones and Jeremy Straub. “Simulation and Analysis of Self-Replicating Robot Decision-Making Systems”. In: *Computers* 10.1 (2021), p. 9. DOI: 10.3390/computers10010009 (cit. on pp. 17, 18).

[Jon21] Andrew Burkhard Jones. “Decision-Making for Self-Replicating 3D Printed Robot Systems”. Ph.D. Dissertation. North Dakota State University, 2021 (cit. on p. 16).

[Abd+22] Amira Abdel-Rahman et al. “Self-replicating hierarchical modular robotic swarms”. In: *Communications Engineering* 1 (2022), p. 35. DOI: 10.1038/s44172-022-00034-3 (cit. on pp. 29, 30, 39).

[Chi22] Gregory S. Chirikjian. “Entropy, Symmetry, and the Difficulty of Self-Replication”. In: *arXiv preprint arXiv:2202.02938* (2022) (cit. on pp. 15, 23).

[Sla+24] J. Tanner Slagel et al. “A Formal Verification Framework for Runtime Assurance”. In: *Proceedings of NASA Formal Methods (NFM 2024)*. Lecture Notes in Computer Science. Mountain View, CA, USA: Springer, June 2024, pp. 322–328. DOI: 10.1007/978-3-031-60698-4_19 (cit. on p. 13).

A Appendix

A.1 NuSMV models

A.1.1 robot_structure.smv

Trigonometric modules

```
MODULE sin(angle)
DEFINE
    val:= case
    angle = 0  :   0;   angle = 1  :  17;       angle = 2  :  34;
    angle = 3  : 50;    angle = 4  :  64;       angle = 5  :  77;
    angle = 6  :  87;   angle = 7  : 94;        angle = 8  :  98;
    angle = 9  : 100;   angle = 10 :  98;       angle = 11 : 94;
    angle = 12 :  87;   angle = 13 :  77;       angle = 14 :  64;
    angle = 15 : 50;    angle = 16 :  34;       angle = 17 :  17;
    angle = 18 :   0;   angle = 19 :-17;        angle = 20 :-34;
    angle = 21 :-50;    angle = 22 :-64;        angle = 23 :-77;
    angle = 24 :-87;    angle = 25 :-94;        angle = 26 :-98;
    angle = 27 :-100;   angle = 28 :-98;        angle = 29 :-94;
    angle = 30 :-87;    angle = 31 :-77;        angle = 32 :-64;
    angle = 33 :-50;    angle = 34 :-34;        angle = 35 :-17;
    TRUE : 0;
    esac;

MODULE cos(angle)
DEFINE
    val:= case
    angle = 0  : 100;   angle = 1  :  98;       angle = 2  :  94;
    angle = 3  : 87;    angle = 4  :  77;       angle = 5  :  64;
    angle = 6  :  50;   angle = 7  : 34;        angle = 8  :  17;
    angle = 9  :   0;   angle = 10 :-17;        angle = 11 :-34;
    angle = 12 :-50;    angle = 13 :-64;        angle = 14 :-77;
    angle = 15 :-87;    angle = 16 :-94;        angle = 17 :-98;
    angle = 18 :-100;   angle = 19 :-98;        angle = 20 :-94;
    angle = 21 :-87;    angle = 22 :-77;        angle = 23 :-64;
    angle = 24 :-50;    angle = 25 :-34;        angle = 26 :-17;
    angle = 27 :   0;   angle = 28 :  17;       angle = 29 :  34;
    angle = 30 :  50;   angle = 31 : 64;        angle = 32 :  77;
    angle = 33 :  87;   angle = 34 :  94;       angle = 35 : 98;
    TRUE : 0;
```

block_base module

```
MODULE block_base(px, py, pz, xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz, L)
DEFINE
  xhx2 := xhx;
  xhy2 := xhx;
  xhz2 := xhz;

  yhx2 := yhx;
  yhy2 := yhx;
  yhz2 := yhz;

  zhx2 := zhx;
  zhy2 := zhy;
  zhz2 := zhz;

  px2 := px;
  py2 := py;
  pz2 := pz;
```

block_yaw module

```
MODULE block_yaw(px, py, pz,
                 xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz,
                 L, initYaw)
VAR
  yaw : 0..35;
  cy : cos(yaw);
  sy : sin(yaw);

DEFINE
  SCALE := 100;

  cos_yaw := cy.val;    -- [-100..100]
  sin_yaw := sy.val;

  xhx2 := (xhx * cos_yaw - xhy * sin_yaw) / SCALE;
  xhy2 := (xhx * sin_yaw + xhy * cos_yaw) / SCALE;
  xhz2 := xhz;

  yhx2 := (yhx * cos_yaw - yhy * sin_yaw) / SCALE;
  yhy2 := (yhx * sin_yaw + yhy * cos_yaw) / SCALE;
  yhz2 := yhz;

  zhx2 := zhx;
  zhy2 := zhy;
  zhz2 := zhz;

  px2 := px + (zhx2 * L) / SCALE;
  py2 := py + (zhy2 * L) / SCALE;
  pz2 := pz + (zhz2 * L) / SCALE;


ASSIGN
  init(yaw) := initYaw;
  next(yaw) := { yaw, (yaw+1) mod 36, (yaw+35) mod 36 };
```

block_pitch module

```
MODULE block_pitch(px, py, pz,
                   xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz,
                   L, initPitch)
VAR
  pitch : 0..35;

  cp : cos(pitch);
  sp : sin(pitch);

DEFINE
  SCALE := 100;

  cos_p := cp.val;    -- [-100..100]
  sin_p := sp.val;

  xhx2 := (xhx * cos_p + zhx * sin_p) / SCALE;
  xhy2 := (xhy * cos_p + zhy * sin_p) / SCALE;
  xhz2 := (xhz * cos_p + zhz * sin_p) / SCALE;

  yhx2 := yhx;  yhy2 := yhy;  yhz2 := yhz;

  zhx2 := (-xhx * sin_p + zhx * cos_p) / SCALE;
  zhy2 := (-xhy * sin_p + zhy * cos_p) / SCALE;
  zhz2 := (-xhz * sin_p + zhz * cos_p) / SCALE;

  px2 := px + (zhx2 * L) / SCALE;
  py2 := py + (zhy2 * L) / SCALE;
  pz2 := pz + (zhz2 * L) / SCALE;

ASSIGN
  init(pitch) := initPitch;
  next(pitch) := { pitch, (pitch+1) mod 36, (pitch+35) mod 36 };
```

main module

```
MODULE main

FROZENVAR

    checkX : -10..10;
    checkY : -10..10;
    checkZ : 0..10;
VAR
    block0 : block_base(base_px, base_py, base_pz,
                        base_xhx, base_xhy, base_xhz,
                        base_yhx, base_yhy, base_yhz,
                        base_zhx, base_zhy, base_zhz,
                        L1);
    block1 : block_yaw(block0.px, block0.py, block0.pz,
                        block0.xhx, block0.xhy, block0.xhz,
                        block0.yhx, block0.yhy, block0.yhz,
                        block0.zhx, block0.zhy, block0.zhz,
                        L1, yaw1 );
    block2 : block_pitch(block1.px2, block1.py2, block1.pz2,
                       block1.xhx2, block1.xhy2, block1.xhz2,
                        block1.yhx2, block1.yhy2, block1.yhz2,
```

```
                        block1.zhx2, block1.zhy2, block1.zhz2,
                        L2, pitch2);
    block3 : block_pitch(block2.px2, block2.py2, block2.pz2,
                        block2.xhx2, block2.xhy2, block2.xhz2,
                        block2.yhx2, block2.yhy2, block2.yhz2,
                        block2.zhx2, block2.zhy2, block2.zhz2,
                        L3, pitch3);
    block4 : block_yaw(block3.px, block3.py, block3.pz,
                          block3.xhx, block3.xhy, block3.xhz,
                          block3.yhx, block3.yhy, block3.yhz,
                          block3.zhx, block3.zhy, block3.zhz,
                          L4, yaw4 );
    block5 : block_pitch(block4.px2, block4.py2, block4.pz2,
                        block4.xhx2, block4.xhy2, block4.xhz2,
                        block4.yhx2, block4.yhy2, block4.yhz2,
                        block4.zhx2, block4.zhy2, block4.zhz2,
                        L5, pitch5);
DEFINE
    L1 := 10;
    yaw1    := 0;
    L2 := 10;
    pitch2  := 0;
    L3 := 10;
    pitch3  := 0;
    L4 := 10;
    yaw4    := 0;
    L5 := 10;
    pitch5  := 0;

    base_px := 0;       base_py := 0;       base_pz := 0;

    base_xhx := 100;    base_xhy := 0;      base_xhz := 0;
    base_yhx := 0;      base_yhy := 100;    base_yhz := 0;
    base_zhx := 0;      base_zhy := 0;      base_zhz := 100;

    endX := block3.px2;
    endY := block3.py2;
    endZ := block3.pz2;

    error := 1; -- +-1 (error=2) endX and endY coordinates error while checking
        reachability
    endX_inside_limits := (endX <= (checkX + error) & endX >= (checkX - error));
    endY_inside_limits := (endY <= (checkY + error) & endY >= (checkY - error));
    endZ_inside_limits := (endZ <= (checkZ + error) & endZ >= (checkZ - error));

CTLSPEC
      EF (endX_inside_limits & endY_inside_limits & endZ_inside_limits ) --&
          blocks_above_serface)
```

B Appendix